Exno: BASIC NETWORK COMMANDS.

Aim:

To execute the basic Network commands in windows for network configuration and troubleshooting.

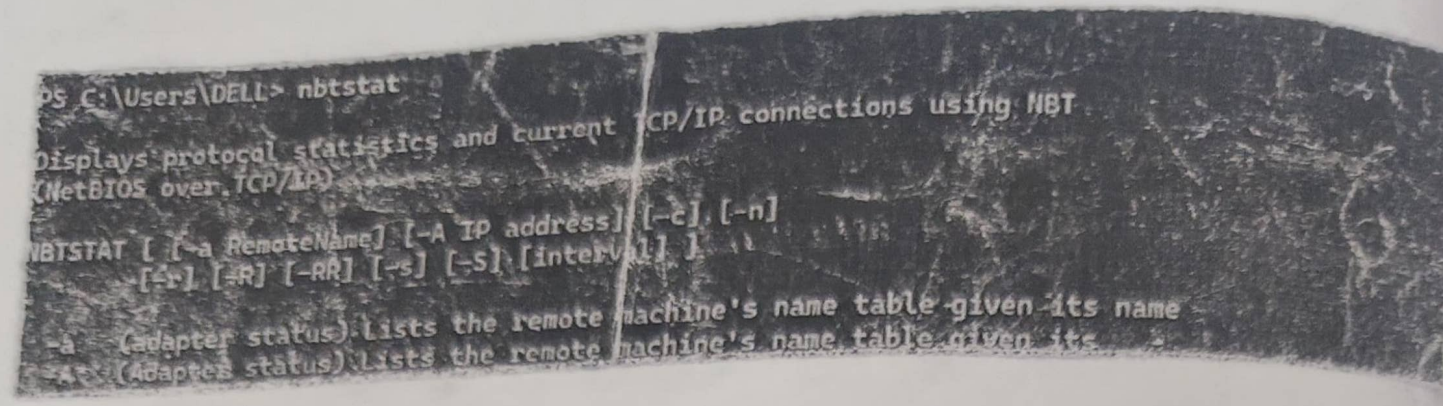
1. arp -a :- ARP is short form of address resolution protocol, It will show the IP computer along with the IP address and MAC address of your router.

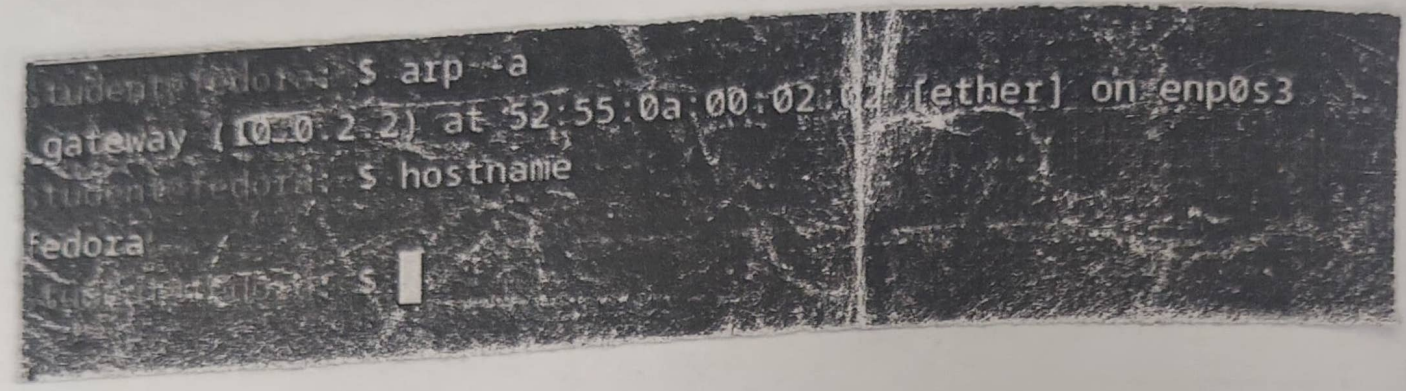
2. Hostname :- This is the simplest of all TCP/IP commands It simply displays the computer.

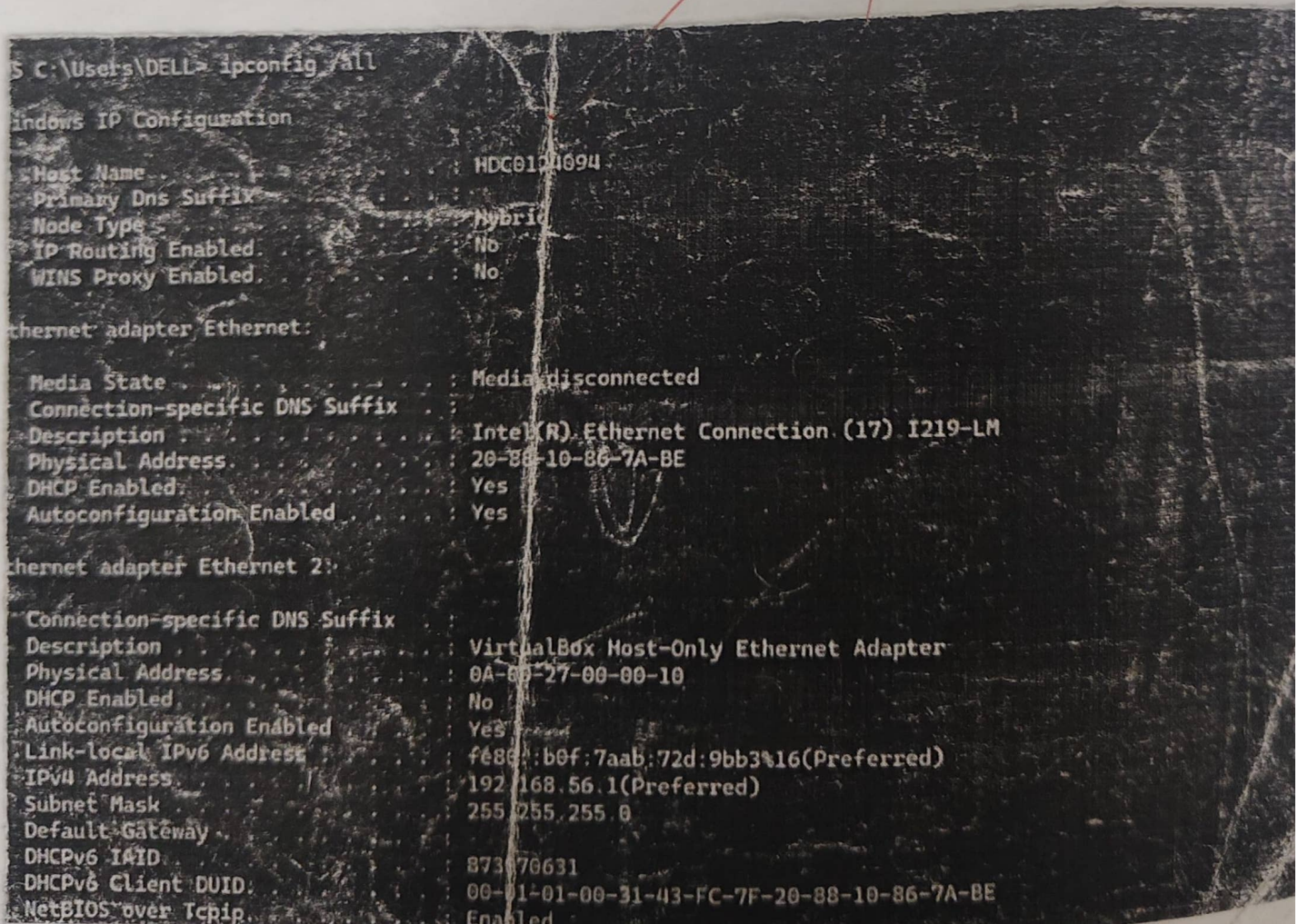
3. ipconfig /all : This command displays detailed configuration information about connection including Router, Gateway, DNS, DHCP, and type of Ethernet and system.

4. nbstat -a : This command helps solve problems with NetBIOS name resolution for NetBIOS over TCP/IP)

5. netstat - netstat displays a variety of statistics about a TCP/IP connections. It is a command line tool for monitoring network connection incoming and outgoing as well as viewing routing tables, interface statistics . eg. netstat -r


S C:\Users\DELL> nbtstat
Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP)
NBTSTAT [[-a RemoteName] [-A IP address] [-c] [-n]
[-r] [-R] [-RR] [-s] [-S] [interval]]
-a (adapter status) Lists the remote machine's name table given its name
-A (Adapter status) Lists the remote machine's name table given its


$ arp -a
gateway (10.0.2.2) at 52:55:0a:00:02:02 [ether] on enp0s3
$ hostname
fedora
$


S C:\Users\DELL> ipconfig /all
indows IP Configuration
Host Name . . . . . : HDC01[illegible]694
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No
thernet adapter Ethernet:
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Description . . . . . : Intel(R) Ethernet Connection (17) I219-LM
Physical Address. . . . . : 20-88-10-86-7A-BE
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
thernet adapter Ethernet 2:
Connection-specific DNS Suffix . :
Description . . . . . : VirtualBox Host-Only Ethernet Adapter
Physical Address. . . . . : 0A-00-27-00-00-10
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::b0f:7aab:72d:9bb3%16(Preferred)
IPv4 Address. . . . . : 192.168.56.1(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . :
DHCPv6 IAID . . . . . : 873[illegible]0631
DHCPv6 Client DUID. . . . . : 00-01-01-00-31-43-FC-7F-20-88-10-86-7A-BE
NetBIOS over Tcpip. . . . . : Enabled

6. nslookup : is a tool used to perform DNS lookups. display DNS detail such as the IP address of a particular computer, the MX domain or the NS servers.

7. pathping : pathping is unique to window's and is basically a combination of Tracert commands. pathping traces the route to the destination address then second test of each router along the way.

8. ping : command is the best way to test connective nodes.

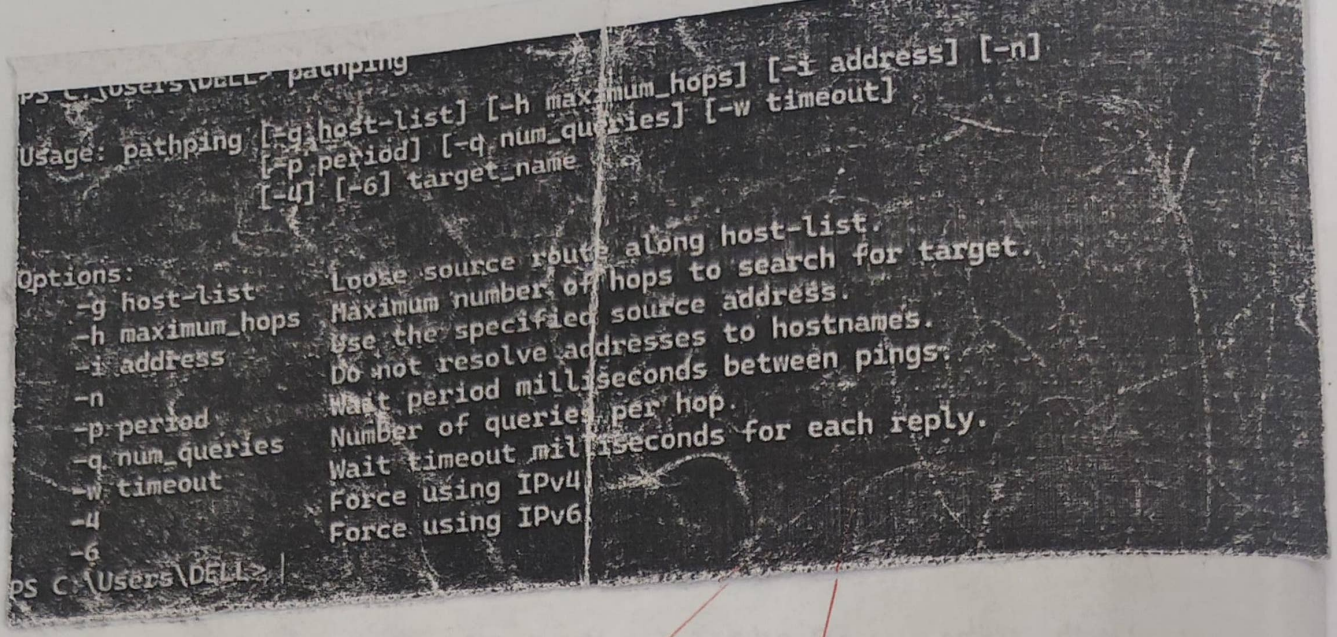
Ping use ICMP to communicate

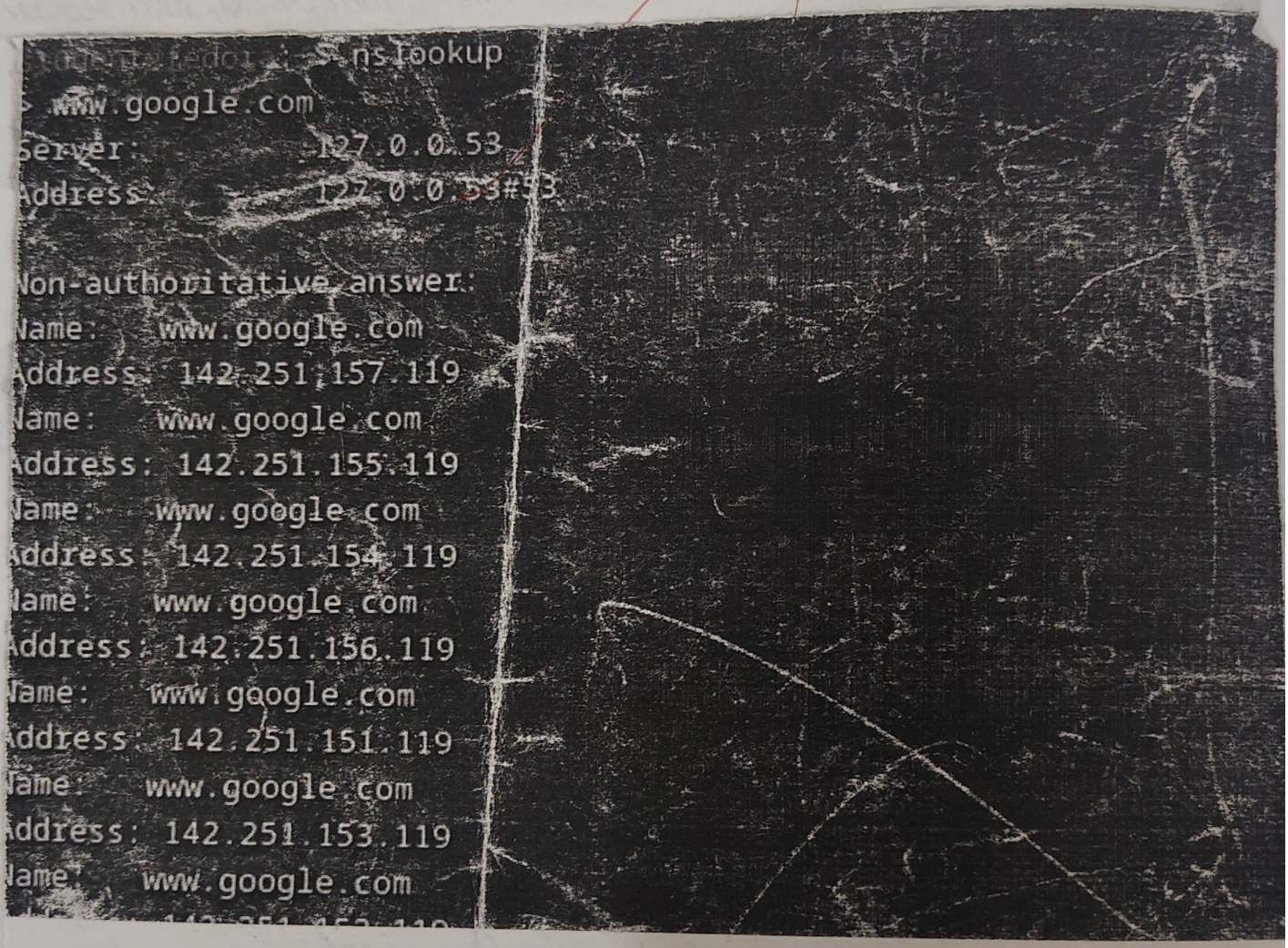
1. #ping hostname (pinglocal host)
2. #ping ip address (ping 4.2.2.2)
3. #ping fully qualified domain name

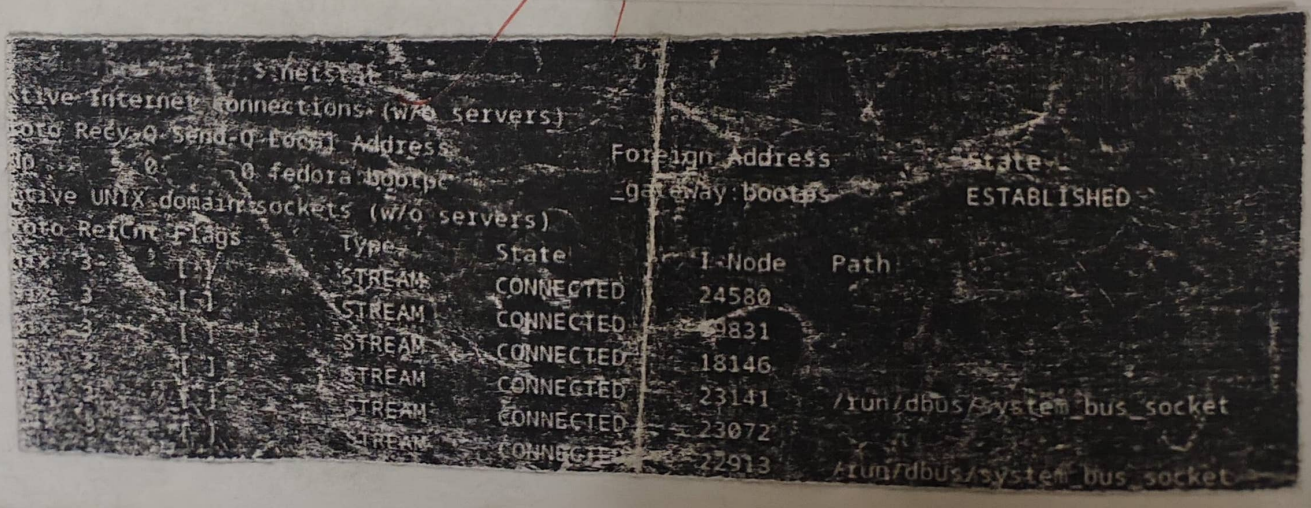
9.) Route :- route command is used to show the IP routing table. It is to setup static routes to specific host or networks via an interface.

Result :-

Thus the basic computer network commands are executed successfully using Linux and windows.


pathping
Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
[-p period] [-q num_queries] [-w timeout]
[-4] [-6] target_name
Options:
-g host-list Loose source route along host-list.
-h maximum_hops Maximum number of hops to search for target.
-i address Use the specified source address.
-n Do not resolve addresses to hostnames.
-p period Wait period milliseconds between pings.
-q num_queries Number of queries per hop.
-w timeout Wait timeout milliseconds for each reply.
-4 Force using IPv4
-6 Force using IPv6


nslookup
www.google.com
Server: 127.0.0.53
Address: 127.0.0.53#53
Non-authoritative answer:
Name: www.google.com
Address: 142.251.157.119
Name: www.google.com
Address: 142.251.155.119
Name: www.google.com
Address: 142.251.154.119
Name: www.google.com
Address: 142.251.156.119
Name: www.google.com
Address: 142.251.151.119
Name: www.google.com
Address: 142.251.153.119
Name: www.google.com


netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address Foreign Address State
tcp 0 0 fedora:bootpc _gateway:bootps ESTABLISHED
Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags Type State I-Node Path
unix 3 [ ] STREAM CONNECTED 24580
unix 3 [ ] STREAM CONNECTED 19831
unix 3 [ ] STREAM CONNECTED 18146
unix 3 [ ] STREAM CONNECTED 23141 /run/dbus/system_bus_socket
unix 3 [ ] STREAM CONNECTED 23072
unix 3 [ ] STREAM CONNECTED 22913 /run/dbus/system_bus_socket

Exno: 2

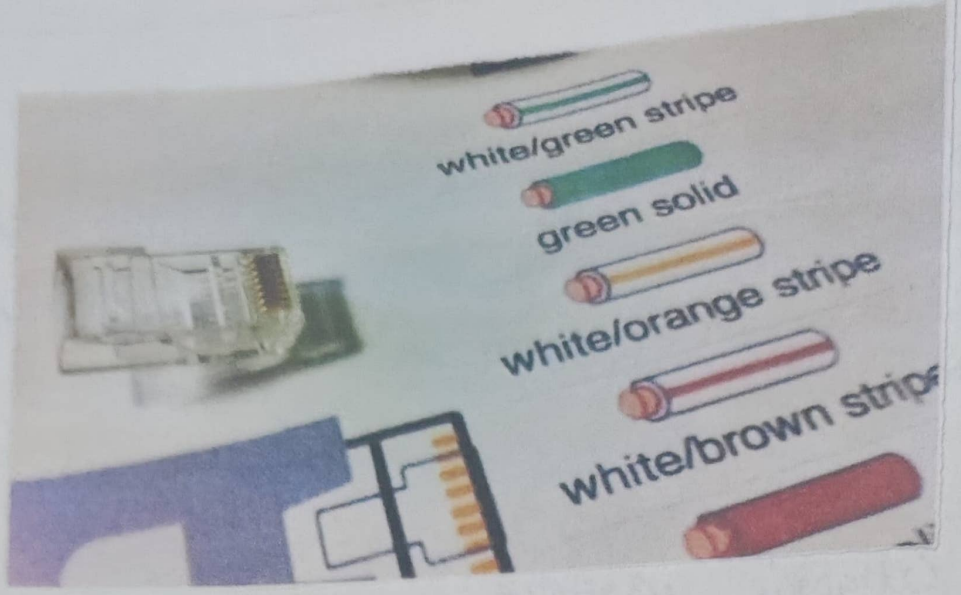
DATE: 23.7.26

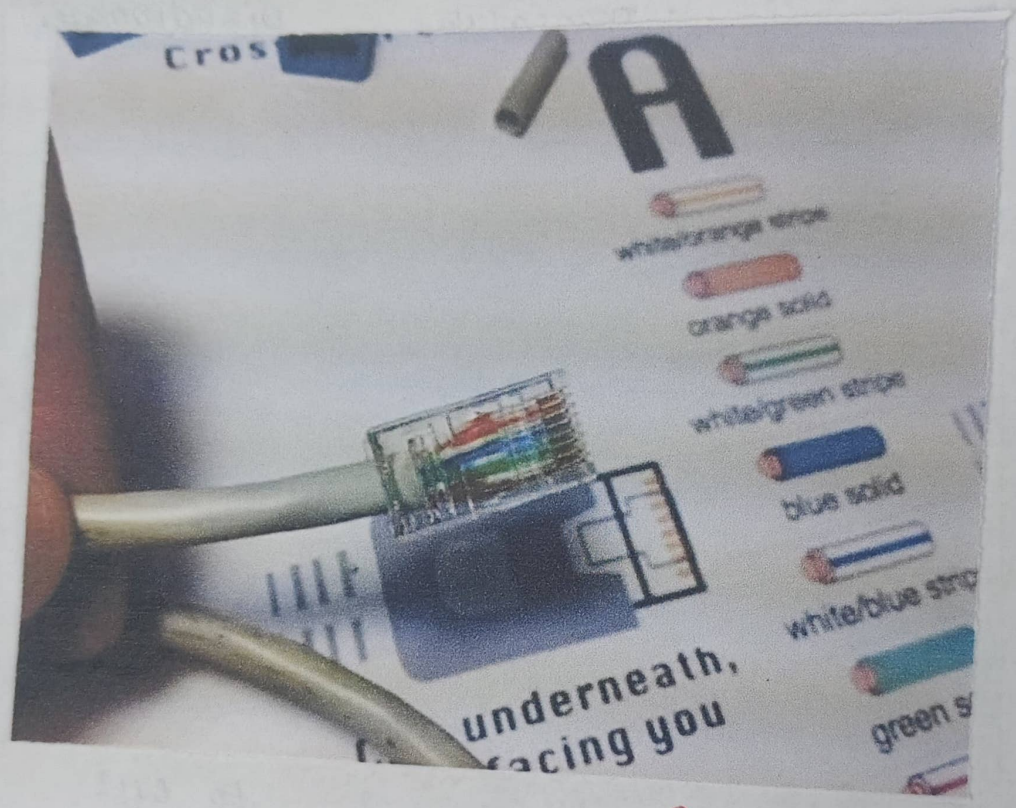
STUDY OF DIFFERENT TYPES OF NETWORK CABLES.

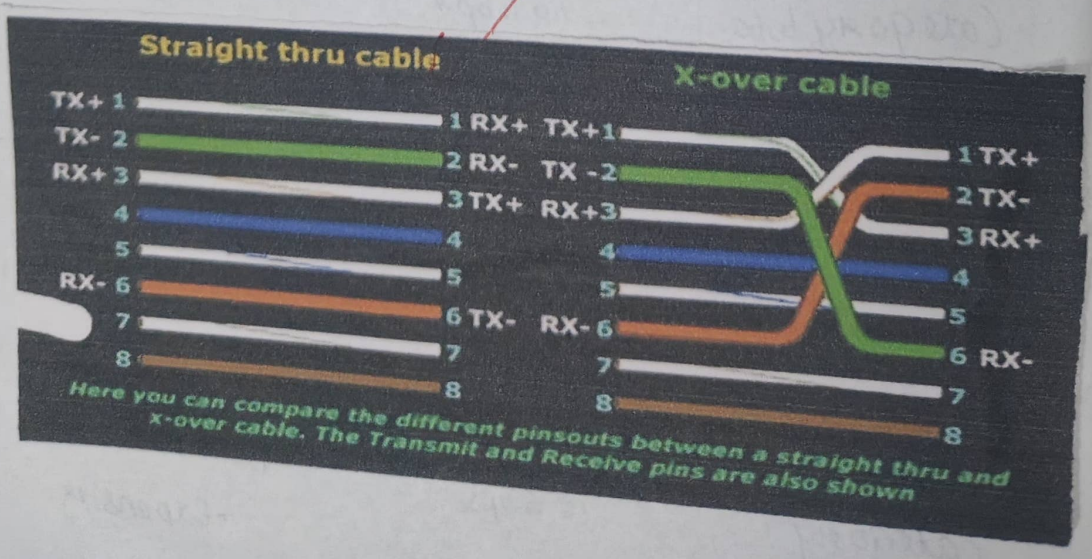
Aim:-

To study the different types of network cables used in computer networking and understand their purpose and applications.

Cable Type	Category	Maximum Data Transmission	Advantages/ Disadvantages	Uses.
UTP	Category 3 Category 5 Category 5e	10bps Upto 100 Mbps 1 Gbps	Advantage • Cheaper in cost • Easy to install Disadvantages • More prone to EMI	10Base-T Ethernet Fast Ethernet, Gigabit Ethernet
STP	Category 6, 6a	10 Gbps	Advantages • Shielded • Faster than UTP • Less susceptible to noise.	Gigabit Ethernet, 10G Ethernet
SSTP	category 7	10 Gbps	Disadvantage • Expensive • Greater	Gigabit Ethernet.


white/green stripe
green solid
white/orange stripe
white/brown stripe


A
orange solid
blue solid
underneath,


Straight thru cable
X-over cable
TX+ 1
TX- 2
RX+ 3
4
5
RX- 6
7
8
1 RX+
2 RX-
3 TX+
4
5
6 TX-
7
8
TX+1
TX -2
RX+3
RX- 6
1 TX+
2 TX-
3 RX+
4
5
6 RX-
7
8
Here you can compare the different pinsouts between a straight thru and x-over cable. The Transmit and Receive pins are also shown

Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	• High bandwidth • Immune to interference • Low loss bandwidth • versatile.	Sp

Step 1: To start construction of the device, begin by threading shields onto the cable.

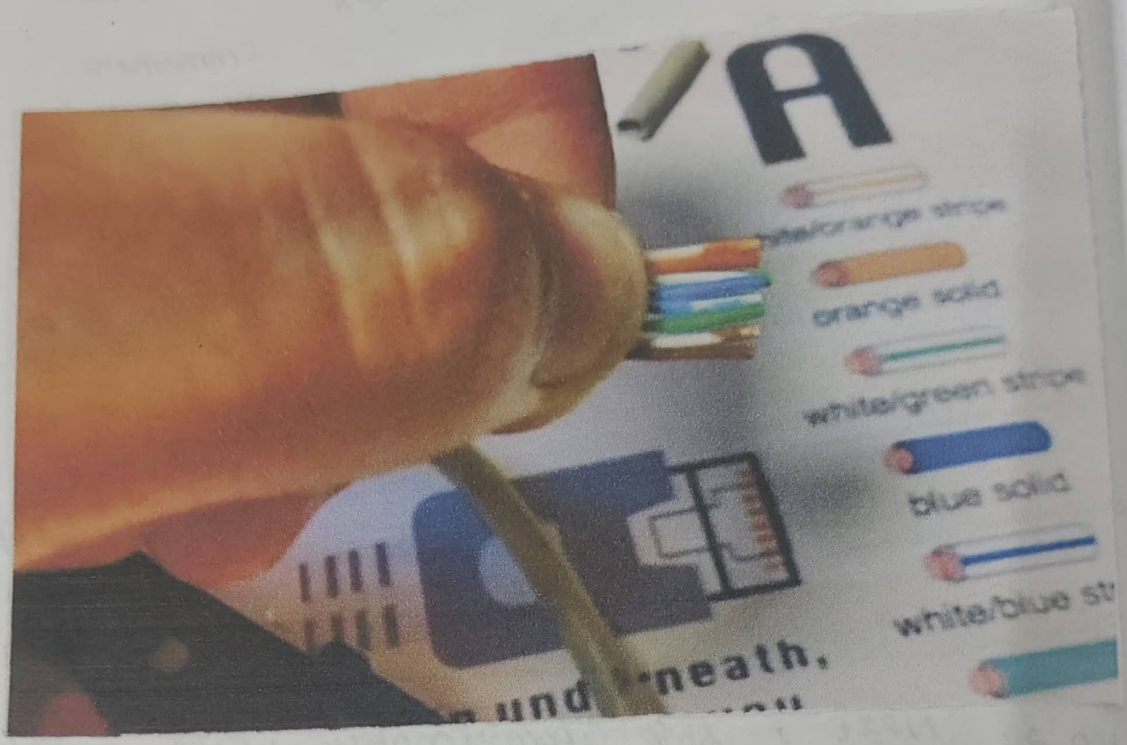
Step 2: Next, strip approximately 1.5cm of cable shielding from both ends. the crimping tool has a round area to complete this task.

Step 3: After, you will need to untangle the wires there should be four "twisted pairs". Referencing back to the sheet.

Step 4: once the order is correct, bunch them together in a line and if there are any that stick out farther than others. The difficult aspect is placing the into the RJ45 plug.

Step 5

Step 6


A
orange solid
white/green stripe
blue solid
white/blue st
neath,

Step 5: Next, push the cable right in. the notch at the end of the plug needs to be just over the cable sheilding.

Step 6: After the wires are securely sitting inside the plug. insert it into the crimping tool and push down.

Step 7: Lastly repeat for the other end using diagram A. TO test it plug it in and attempt to connect two devices directly.

Result:

Thus, different types of network cables were studied successfully, and straigh Through and Cross over Ethernet cables were prepared and tested successfully.

Step 5: Next, push the cable right in. the notch at the end of the plug needs to be just over the cable shielding.

Step 6: After the wires are securely sitting inside the plug. insert it into the crimping tool and push down.

Step 7: Lastly repeat for the other end using diagram A. TO test it plug it in and attempt to connect two devices directly.

Result:

Thus, different types of network cables were studied successfully, and straigh Through and Cross over Ethernet cables were prepared and tested successfully.

Ex: 3 Understanding the Cisco Packet Tracer Environment

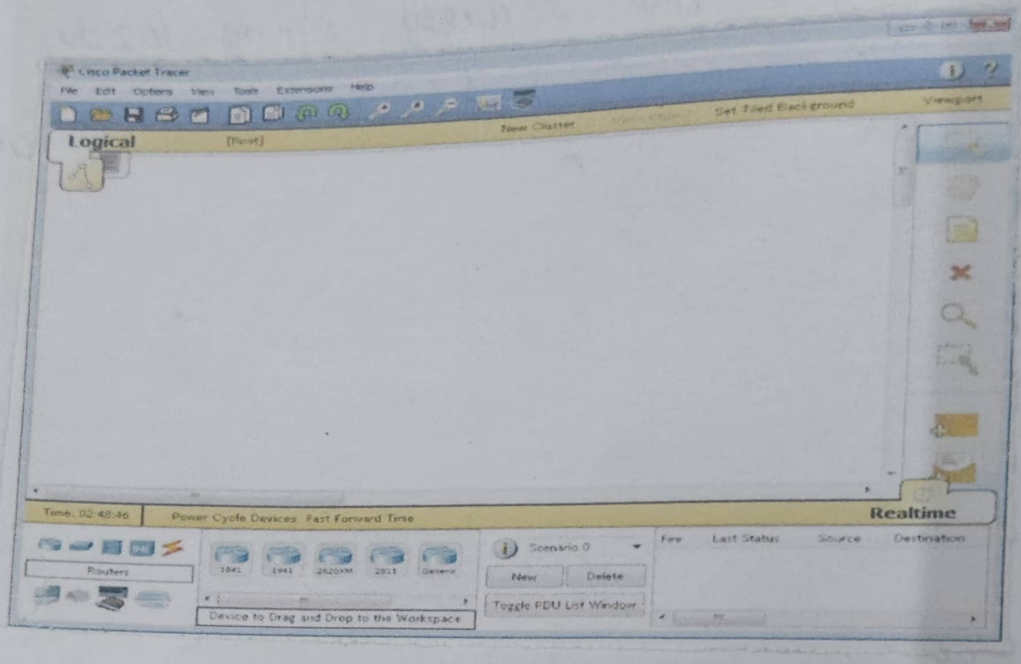
Aim:-

To understand the Cisco Packet Tracer environment and design a simple computer network using its interface and networking components.

Steps:-

- Open the Cisco Packet Tracer application
- Observe the user interface, including the Menu Bar, Main Toolbar, Workspace, Common Tools Bar, Network component Box, and Realtime/Simulation tabs.
- From the Network component Box, select the required network devices such as PCs, Hub and Switch.
- Drag and drop the devices into the workspace.
- Connect the devices using copper straight. Through cables.

Configure IP addresses for the PCs using the Desktop → IP Configuration option.


Cisco Packet Tracer
File Edit Options View Tools Extensions Help
Logical
[Root]
Time: 00:48:46
Power Cycle Devices Fast Forward Time
Realtime
Routers
Device to Drag and Drop to the Workspace
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination

- verify that all links are active.
- Use the Add Simple PDU tool to send a packet between two PCs.
- Observe packet transmission in Realtime mode and Stimulation Mode.
- Save the network topology.

Result:-

The Cisco Packet Tracer environment was successfully explored and a simple network was designed. The interface components, device connections, IP configuration and packet transmission were successfully understood using Realtime and Stimulation modes.

DATE: 23-7-26

Ex: 3a. HUB.

Aim:

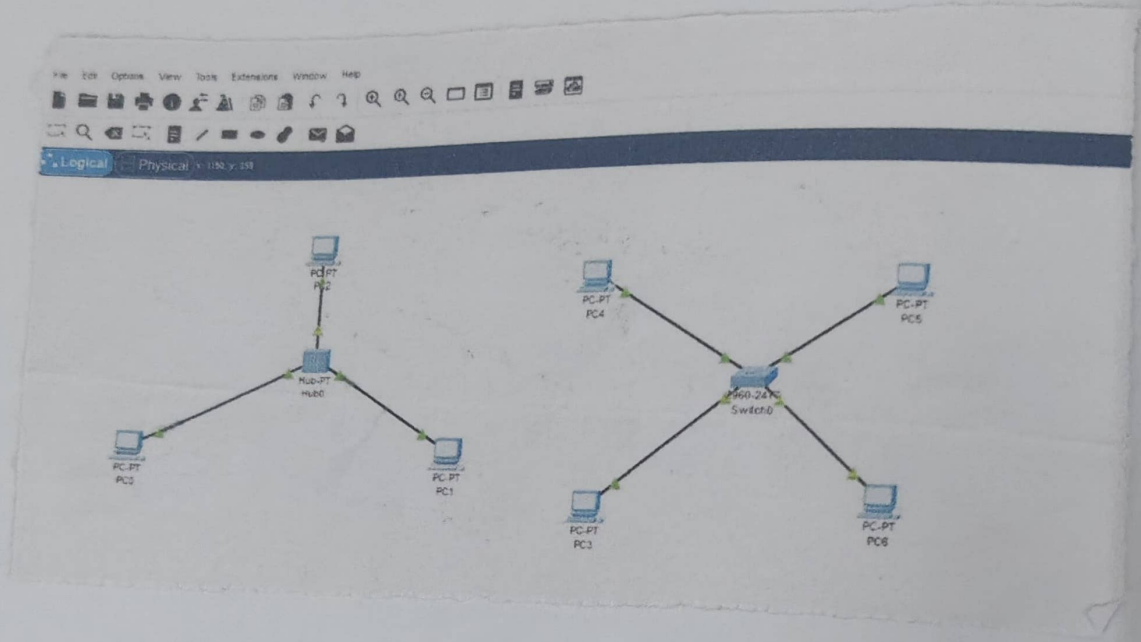
To study the behavior of a Hub by observing how it forwards data packets using Cisco Packet Tracer

Steps:

- Open Cisco Packet Tracer.
- Drag and place 1 Hub and 4 Generic PCs into the workspace.
- Connect all PCs to the Hub using Copper Straight-Through cables
- Configure IP addresses for all PCs in the same network
- Select Add Simple PDU from the common tools bar.
- Send a packet from one PC to another PC.
- Switch to simulation mode and observe how the hub forwards the packet.

Result:

The Hub broadcasts the received packet to all connected devices. Every PC receives the packet, but only the destination PC accepts it while others discard it.


File Edit Options View Tools Extensions Window Help
Logical
Physical
PC-PT PC2
Hub-PT Hub0
PC-PT PC0
PC-PT PC1
PC-PT PC4
PC-PT PC5
2960-24TT Switch0
PC-PT PC3
PC-PT PC6

DATE: 23.7.24 EX: 3(b) SWITCH.

AIM:

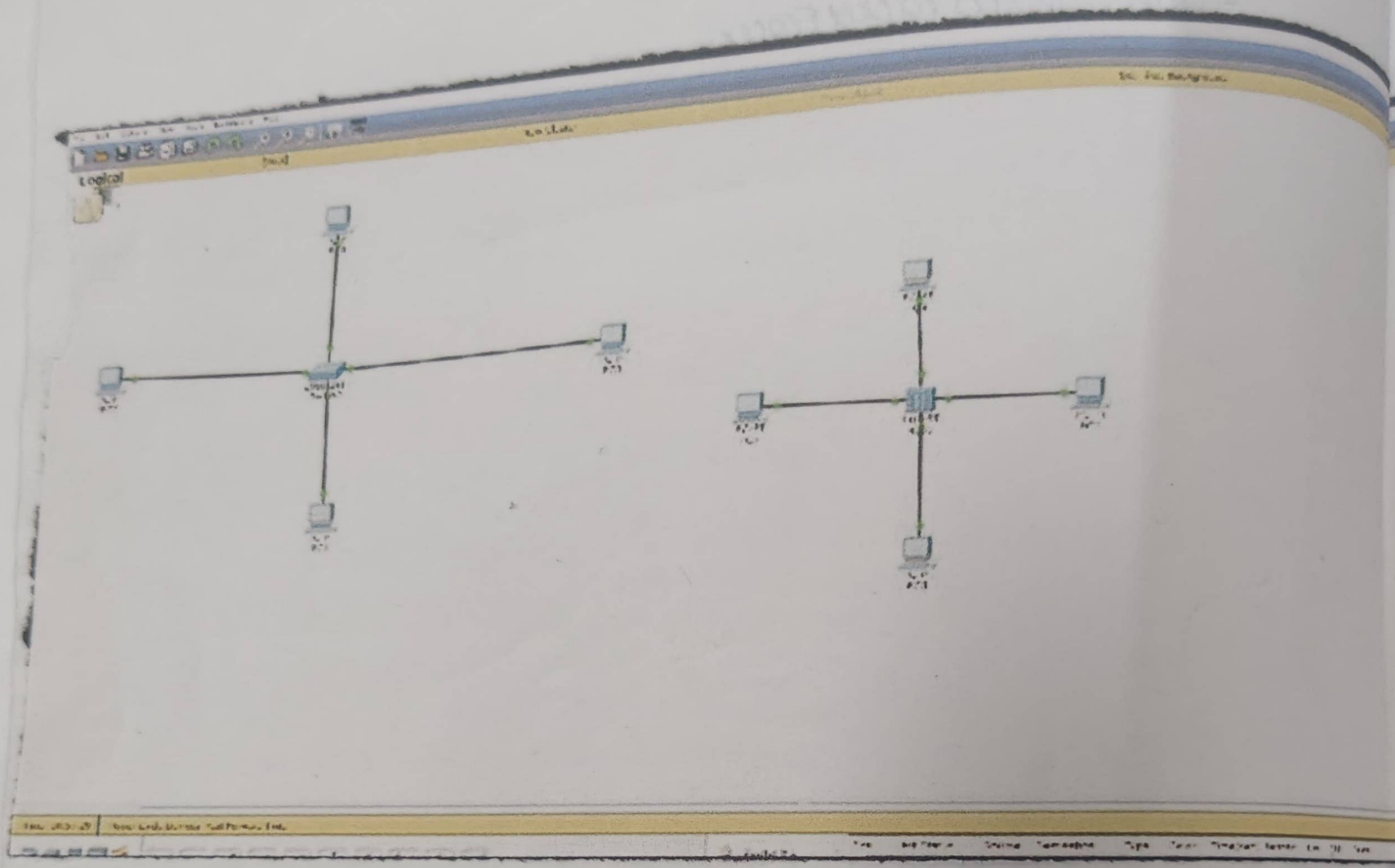
To study the behavior of a switch by observing how it forwards data packets using Cisco Packet tracer.

STEPS:

- Open Cisco Packet Tracer.
- Drag and place 1 Switch and 4 Generic PCs into the workspace.
- Connect all PCs to the switch using Copper Straight-Through cables.
- Configure IP addresses for all PCs in the same network.
- Select Add Simple PDU from the common Tools bar.
- Send a packet from one PC to another PC.
- Switch to Simulation mode and observe how the switch forwards the packet.

Result:

The switch forwards the packet only to the destination PC by using its MAC address table. This reduces unnecessary network traffic and improves communication efficiency.


Logical

DATE: 23.7.26

Ex: 3 b2 HUB and Switch.

Aim:-

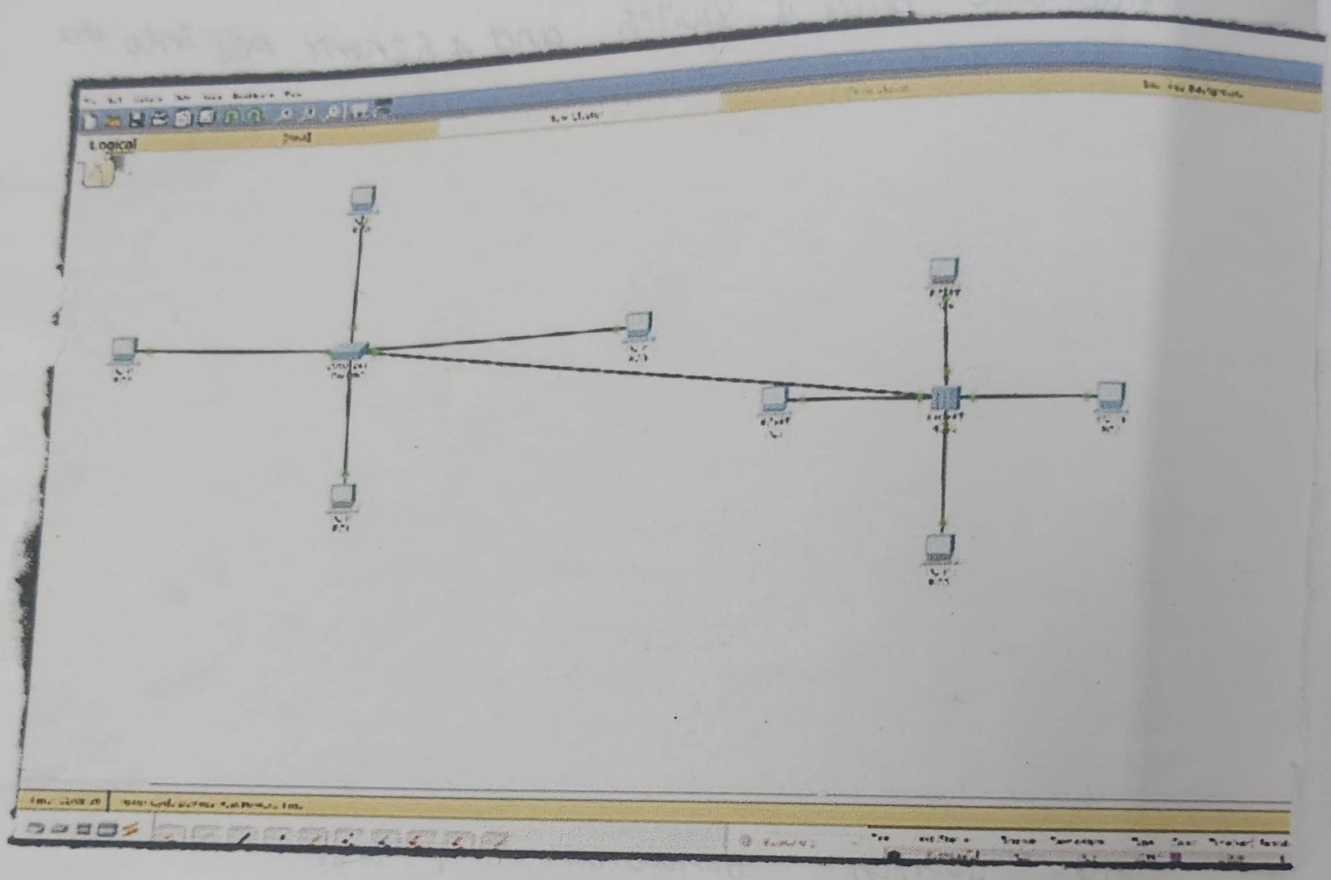
To compare the packet forwarding behavior of a Hub and a Switch using Cisco Packet-Tracer.

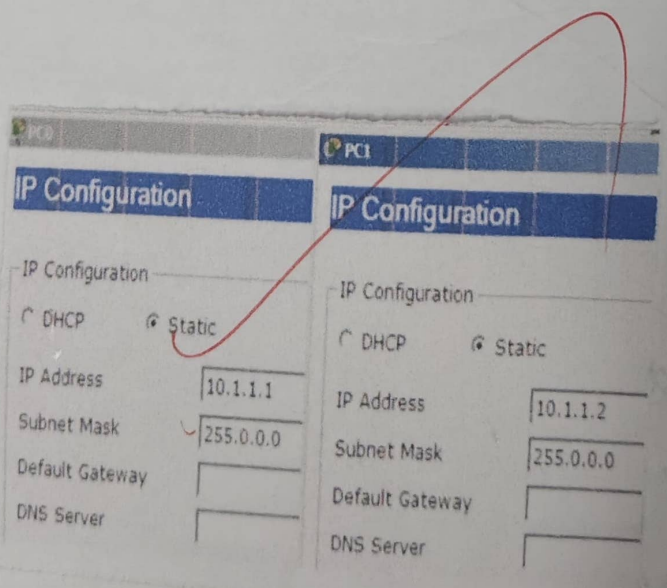
Steps:

- Create one network using a HUB and another using a switch, each connected to 4 PCs.
- Configure IP addresses for all PCs in both networks.
- Send a simple PDU between two PCs in each network.
- Observe packet transmission in simulation mode.
- Compare how the hub and switch forward packets.

Result:

The hub broadcasts packets to all connected devices, whereas the switch forwards packets only to the intended destination. Therefore, the switch provides better performance, reduced network traffic, and improved efficiency compared to the hub.


Logical


PC0
IP Configuration
IP Configuration
DHCP
Static
IP Address
10.1.1.1
Subnet Mask
255.0.0.0
Default Gateway
DNS Server
PC1
IP Configuration
IP Configuration
DHCP
Static
IP Address
10.1.1.2
Subnet Mask
255.0.0.0
Default Gateway
DNS Server

Ex:no 4 LAN using a Switch and Ethernet cables.

Aim:-

To setup and configure a LAN (Local area network) using a switch and Ethernet cables.

What is CAN?

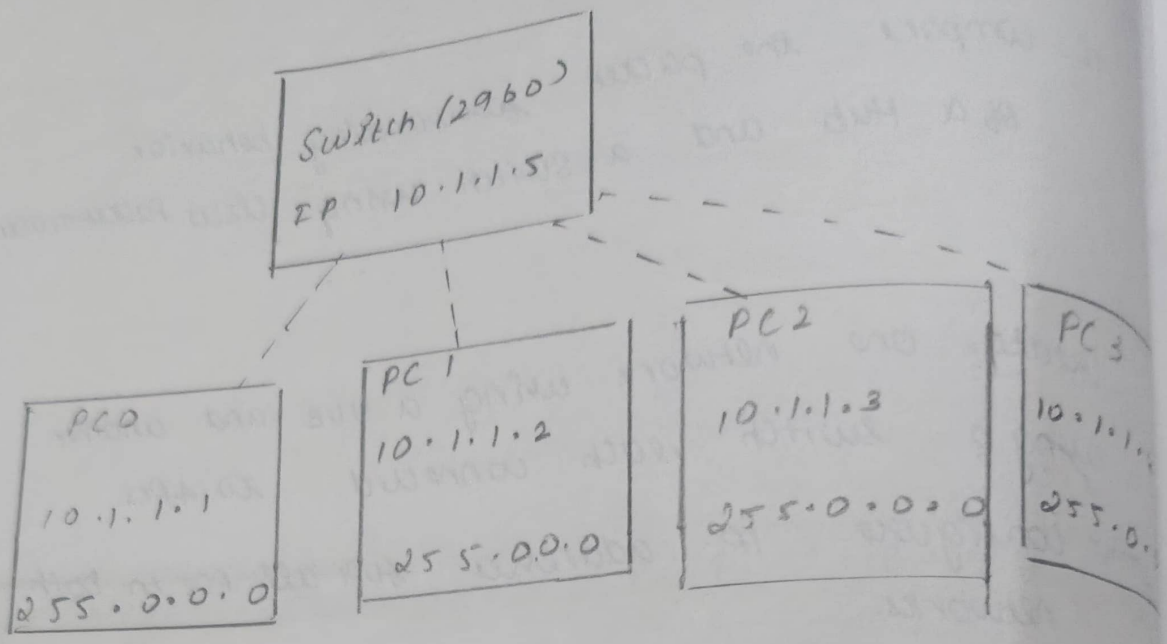
- A Local Area Network refers to a network that connects devices within a limited area, such as an office building, school, or home. It enables users to share resources, including data, printers, and internet access. Each connected device on a LAN switch can communicate directly with each other, allowing for fast and secure data transfer.

Steps:

1. Plan and Design an appropriate network topology taking into account network requirements and equipment location.

2. You can take 4 Computers, a Switch with 8, 16, or 24 ports which is sufficient for networks of these sizes and 4 Ethernet cables.

3. Connect your computers to network switch via an Ethernet cable, which is as simple as plugging one end of the Ethernet


Switch (2960)
IP 10.1.1.5
PC0
10.1.1.1
255.0.0.0
PC 1
10.1.1.2
255.0.0.0
PC 2
10.1.1.3
255.0.0.0
PC 3
10.1.1.
255.0.

4. Assign IP address to your PCs

1. Log on to the client computer as Administrator or as owner.

2. Click Network and Internet connections.

3. Right click Local Area Connection / Ethernet → Go to Properties → Select Internet Protocol → Click on Properties.

5. configure a network switch:

1. Connect your computer to the switch. TO access the switch's web interface. you will need to connect your computer to the switch using an Ethernet cable.

2. Log into web interface: Open a web browser and enter the IP address of the switch in the address bar.

3. configure basic settings: Once you're logged in, you will be able to configure basic settings for the switch.

4. Assign IP address as: 10.1.1.5 subnet mask 255.0.0.0.

Step 6:
check the connectivity between switch and other machine by using ping command in the command prompt of the device.

Step7: Select a folder → goto properties → click sharing tab → share it with everyone on the same LAN.

Step8: try to access the shared folder from other computers of the network.

Result:-

A LAN was successfully configured using a switch and ethernet cables. IP addres were assigned correctly. communication was verified using the ping command, and fi sharing was successfully demonstrated computer the LAN.

Exno:05
Date : 30/7/25

Experiment on Packet capture tool wireshark

Aim:-
TO understand the features of wireshark as a packet capture tool and understand encapsulation of information at various layers.

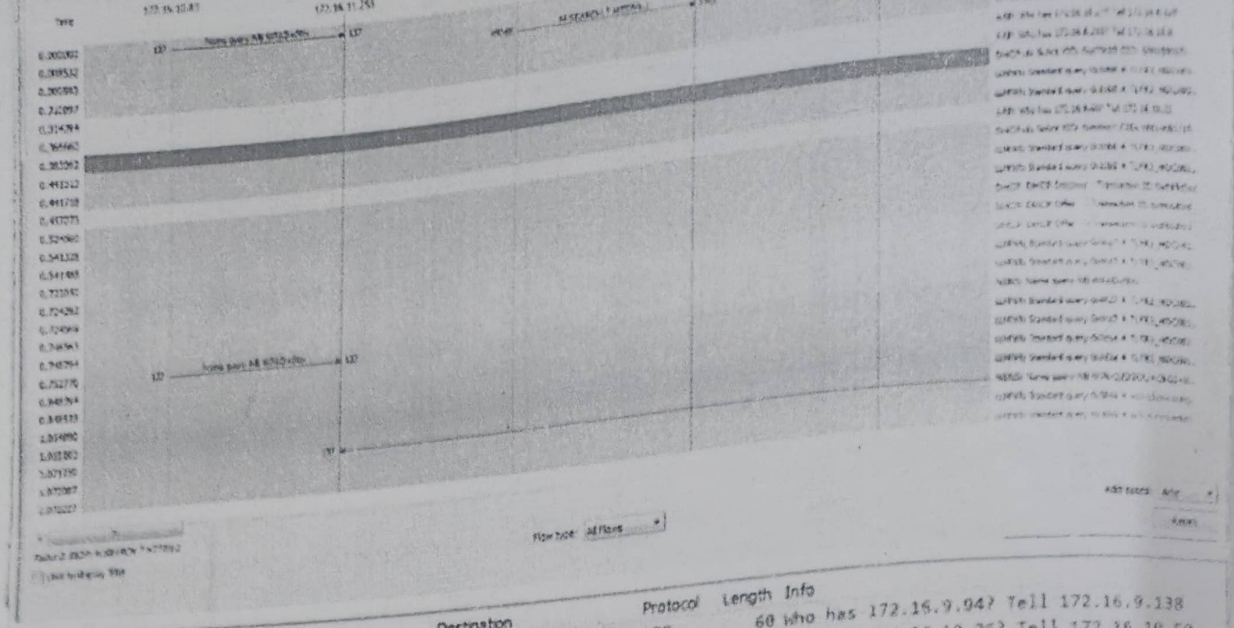
Packet Sniffer:-
* capture packets sent and received by the computer
* Displays protocol information of captured packets.
* works passively.

Packet Sniffer Tools:-
* TCP dump - command line packet capture too.
* wire shark - Graphical packet analyzes.

Description:-
wireshark is a network protocol analyzes that captures and displays networks packets in real time.

It helps monitor, analyze, troubleshoot and inspect network traffic.

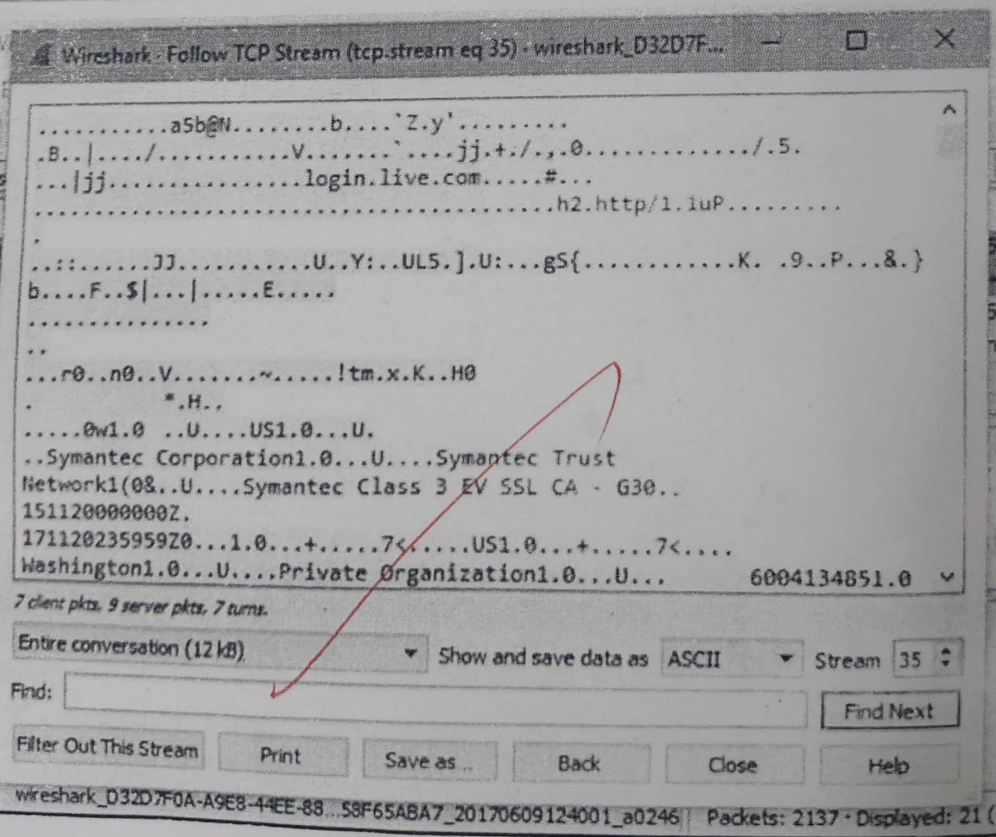
Uses of Network:-
* capture network traffic.
* Analyze protocols
* Applies display filters



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	Pegatron_e0:87:9e	Broadcast	ARP	60	Who has 172.16.9.94? Tell 172.16.9.138
2	0.000180	RealtekS_55:2c:b8	Broadcast	ARP	60	Who has 172.16.10.36? Tell 172.16.10.50
3	0.000294	RealtekS_55:2c:b8	Broadcast	ARP	60	Who has 172.16.11.36? Tell 172.16.10.50
4	0.000295	RealtekS_55:2c:b8	Broadcast	ARP	60	Who has 172.16.8.37? Tell 172.16.10.50
5	0.000296	RealtekS_55:2c:b8	Broadcast	ARP	60	Who has 172.16.9.37? Tell 172.16.10.50
6	0.000296	RealtekS_55:2c:b8	Broadcast	ARP	60	Who has 172.16.11.37? Tell 172.16.10.50
7	0.001460	fe80::4968:12a7:5e3…	ff02::1:3	LLMNR	95	Standard query 0xae2b A TLFL3-HDC101701
8	0.001622	172.16.8.95	224.0.0.252	LLMNR	75	Standard query 0x28c0 AAAA TLFL3-HDC101701
9	0.001623	172.16.8.95	224.0.0.252	LLMNR	75	Standard query 0x28c0 AAAA TLFL3-HDC101701
10	0.001625	fe80::4968:12a7:5e3…	ff02::1:3	LLMNR	95	Standard query 0xae2b A TLFL3-HDC101701

```
▷ Frame 7: 95 bytes on wire (760 bits), 95 bytes captured (760 bits) on interface 0
▷ Ethernet II, Src: Dell_35:10:a8 (50:9a:4c:35:10:a8), Dst: IPv6mcast_01:00:03 (33:33:00:01:00:03)
▷ Internet Protocol Version 6, Src: fe80::4968:12a7:5e36:523e, Dst: ff02::1:3
◢ User Datagram Protocol, Src Port: 62374, Dst Port: 5355
    Source Port: 62374
    Destination Port: 5355
    Length: 41
    Checksum: 0x90e0 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 0]
▷ Link-local Multicast Name Resolution (query)
```

```
0000  33 33 00 01 00 03 50 9a  4c 35 10 a8 86 dd 60 00   33  P  L5  `
0010  00 00 00 29 11 01 fe 80  00 00 00 00 00 00 49 68   )  Ih
0020  12 a7 5e 36 52 3e ff 02  00 00 00 00 00 00 00 00   ^6R>
0030  00 00 00 01 00 03 f3 a6  14 eb 00 29 90 e0 ae 2b   )  +
0040  00 00 00 01 00 00 00 00  00 00 [redacted]
0050  [redacted] 00 01 00 01
```



Applications

* Network troubleshooting.
* Security analysis.
* Protocol debugging.
* Learning network protocols.

Capturing packet:

1.) Open wires shark
2.) Select the network interface.
3.) Click start capture.
4.) Observe packet in real time.
5.) Click stop and save the capture.

wireshark packet:

* Packet list
* Packet details.
* Packet bytes.

color coding:

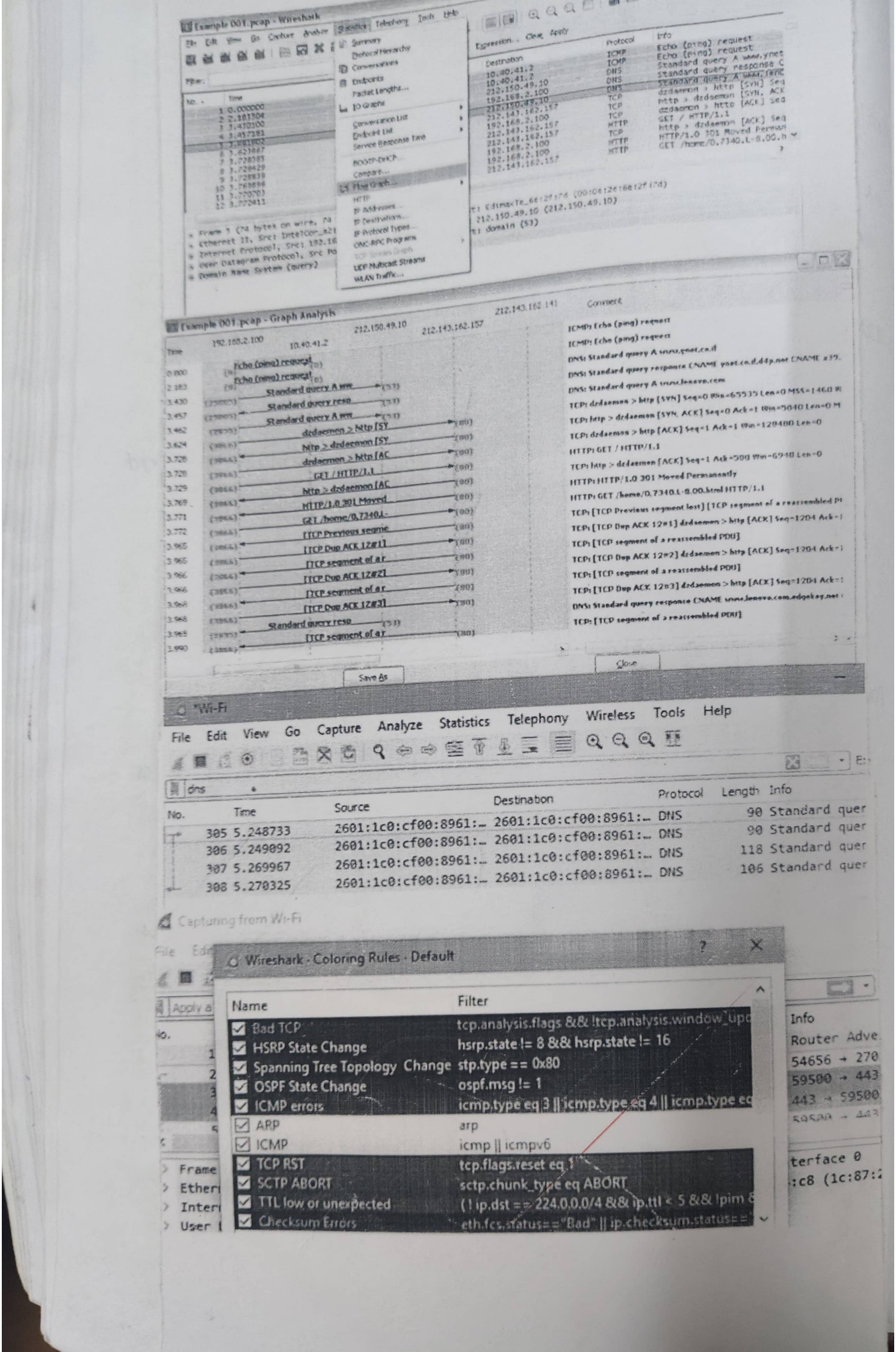
* Purple : TCP
* Blue : VOP
* Black : Error packets.

Filtering packets:

* Enter a filter in the filter bar
* Right click to apply filter

Flow graph:-

Displays communication flow between devices making packing exchange easier to understand.


Example 001.pcap - Wireshark
Example 001.pcap - Graph Analysis
*Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
dns
No. Time Source Destination Protocol Length Info
305 5.248733 2601:1c0:cf00:8961:... 2601:1c0:cf00:8961:... DNS 90 Standard quer
306 5.249092 2601:1c0:cf00:8961:... 2601:1c0:cf00:8961:... DNS 90 Standard quer
307 5.269967 2601:1c0:cf00:8961:... 2601:1c0:cf00:8961:... DNS 118 Standard quer
308 5.270325 2601:1c0:cf00:8961:... 2601:1c0:cf00:8961:... DNS 106 Standard quer
Capturing from Wi-Fi
Wireshark · Coloring Rules · Default
Name
Filter
Bad TCP
tcp.analysis.flags && !tcp.analysis.window_upd
HSRP State Change
hsrp.state != 8 && hsrp.state != 16
Spanning Tree Topology Change
stp.type == 0x80
OSPF State Change
ospf.msg != 1
ICMP errors
icmp.type eq 3 || icmp.type eq 4 || icmp.type eq
ARP
arp
ICMP
icmp || icmpv6
TCP RST
tcp.flags.reset eq 1
SCTP ABORT
sctp.chunk_type eq ABORT
TTL low or unexpected
(! ip.dst == 224.0.0.0/4 && ip.ttl < 5 && !pim
Checksum Errors
eth.fcs.status=="Bad" || ip.checksum.status==
Info
Router Adve
54656 → 270
59500 → 443
443 → 59500
terface 0
:c8 (1c:87:

Create a filter to display only HTTP packets and inspect the packets is procedure

→ Select local Area connection in wireshark

→ Go to capture option.

→ select stop capture automatically after 100 results.

→ Then click start capture.

→ Save the packets.

procedure:

→ select local area connection in wireshark.

→ Go to capture option.

→ Select stop capture automatically after 100 packets.

→ Then click start capture.

→ Save the packets

Result:

Thus the experiments are executed succesfully and experiments on packet capture tool wireshark

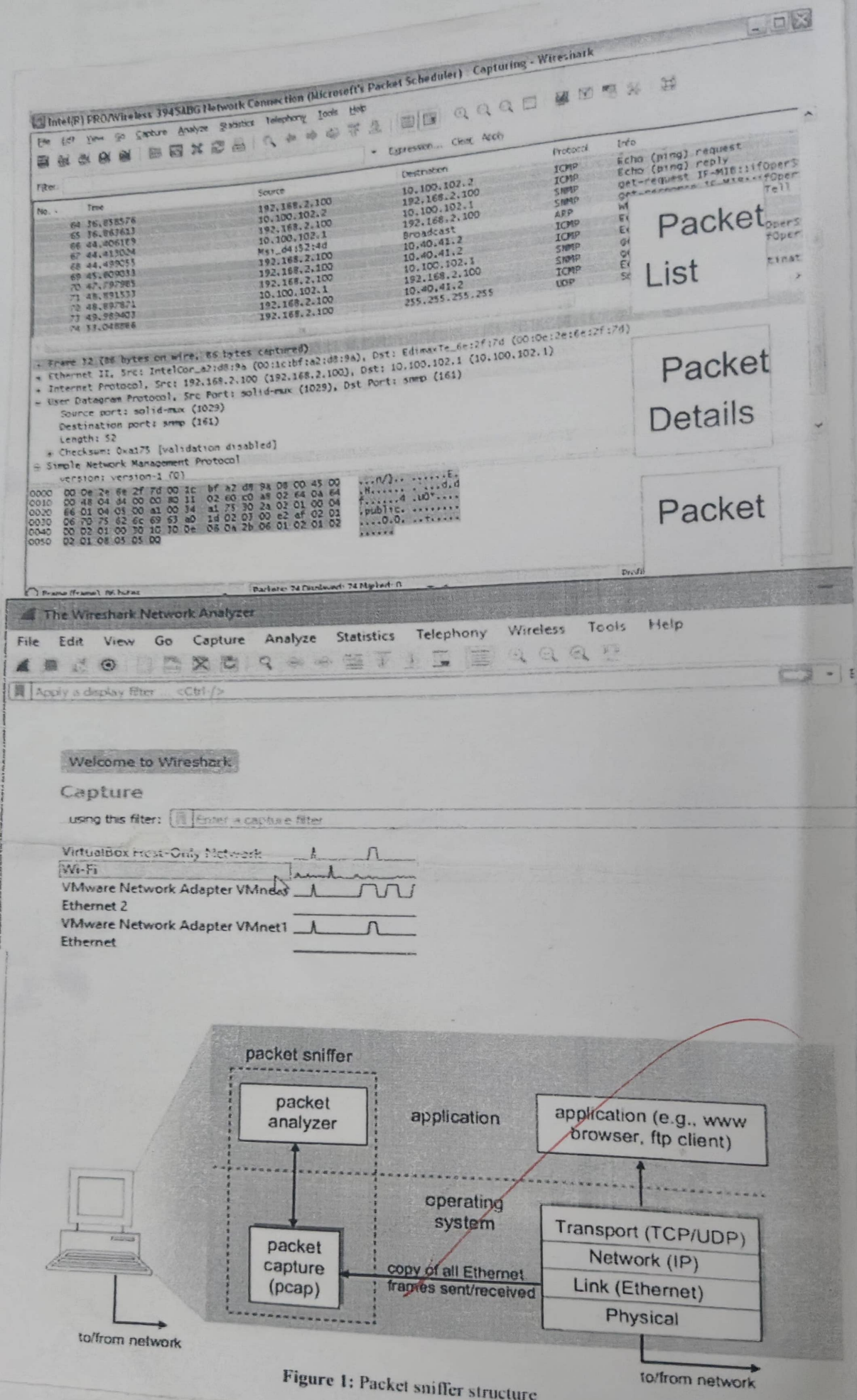


Figure 1: Packet sniffer structure

E: 6.8.26

Exno.6 Error Detection and Correction using Hamming code.

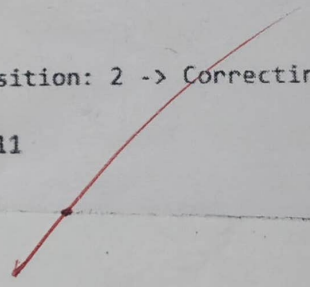
Aim:-

To write a program to implement error detection and correction using the Hamming code.

Algorithm:-

1. Start the program.
2. Read the number of 4-bit blocks (n) forming the input data stream.
3. For each 4-bit data block D = d1 d2 d3 d4. perform Hamming encoding.
   a. Arrange 7 bits with parity bits at positions 1, 2 and 4, and data bits at position 3, 5, 6 and 7.
   b. Compute the parity bits using even parity
      P_1 : d1 XOR d2 XOR d4
      P_2 : d1 XOR d3 XOR d4
      P_4 : d2 XOR d3 XOR d4
4. Stimulate transmission by introducing a single bit error at a chosen bit position in the code word.
5. At the receiver, recompute the check bits from the received code word.
   C1 : P_1 XOR d1 XOR d2 XOR d4
   C2 : D_2 XOR d1 XOR d3 XOR d4
   C4 : P_4 XOR d2 XOR d3 XOR d4

```
... === HAMMING CODE - ERROR DETECTION AND CORRECTION ===

    Enter number of 4-bit data blocks in the stream: 1011

    --- Block 1 ---
    Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
    Original Data Bits              : 1011
    Encoded Hamming Code (7 bits) : 0110011
    Enter bit position to introduce error (1-7, 0 = no error): 3
    Transmitted Code (error at position 3) : 0100011
    Received Code : 0100011
    Checking for errors...
    Error detected at bit position: 3 -> Correcting...
    Corrected Code : 0110011
    Recovered Data Bits : 1011

    --- Block 2 ---
    Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
    Original Data Bits              : 1011
    Encoded Hamming Code (7 bits) : 0110011
    Enter bit position to introduce error (1-7, 0 = no error): 2
    Transmitted Code (error at position 2) : 0010011
    Received Code : 0010011
    Checking for errors...
    Error detected at bit position: 2 -> Correcting...
    Corrected Code : 0110011
    Recovered Data Bits : 1011
```



6. Compute Error Position = C4 C2 C1

7. If Error position = 0. no error has occured.
Else the bit at "Error position" is inverted to correct the error.

8. Extract the original 4 data bits from the corrected 7-bit code word.

9. Repeat step 3 to 8 for all n blocks in the data stream.

10. Stop the program.

PROGRAM:

```
def encode(d):
    """ d: list of 4 bits [d1, d2, d3, d4] → returns 7-bit
    hamming code list positions (1-indexed): 1=p1 2=p2
    3=d1 4=p4 5=d2 6=d3 7=d4 """
    h = [0]*7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h

def detect_and_correct(h):
```

```
else:
    print(f"Transmitted code (no error induced): {to_str(h)}")

print(f"Received code : {to_str(h)}")
print("Checking for errors...")
detect_and_correct(h)
print(f"Corrected code : {to_str(h)}")
rd = [h[2], h[4], h[5], h[6]]
print(f"Recovered Data Bits : {to_str(rd)}")

if __name__ == "__main__":
    main()
```

Result :-

The program to implement error detection and correction using the Hamming code concept was successfully written in Python, executed and verified using a sample input data str

DATE: 6.8.26. EX.NO7 Flow control at Data Link Layer using Sliding Window Protocol.

Aim:-

To write a program to implement flow control at the Data Link Layer using the sliding window Protocol and to stimulate the flow of frames from one node to another.

Algorithm:-

1. Start the program
2. Read the total number of frames to be sent (N), the sender's window size (W), and a frame number to stimulate as lost during transmission
3. Initialize base = 0 and nextSeq = 0
4. Repeat- while base < N:

a.) Determine the current sender window as the range [base, base + w-1].

b Send all frames with sequence number from nextSeq upto (base + w-1), incrementing nextSeq after each frame. Since the window restricts the sender to at most W outstanding frames at any time.

c.) If the lost frames lies within the frames just sent.

i) the receiver discards the lost

```
=== SLIDING WINDOW PROTOCOL - FLOW CONTROL SIMULATION (Go-Back-N) ===

Enter total number of frames to send : 8
Enter sender's window size : 4
Enter frame number to simulate as LOST
(0 to 7, enter -1 for no loss): 5

-----------------------------------------
Sender Window : [Frame 0 - Frame 3]
Node A --> Sending Frame 0
Node A --> Sending Frame 1
Node A --> Sending Frame 2
Node A --> Sending Frame 3
Node B --> Received Frame 0, sending ACK 0
Node B --> Received Frame 1, sending ACK 1
Node B --> Received Frame 2, sending ACK 2
Node B --> Received Frame 3, sending ACK 3
Node A --> ACK 3 received. Sliding window forward.

-----------------------------------------
Sender Window : [Frame 4 - Frame 7]
Node A --> Sending Frame 4
Node A --> Sending Frame 5
(Frame 5 LOST in transit!)
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Frame 5 missing. Discarding out-of-order frames after it.
Node A --> Timeout! Resending window starting from Frame 5.

-----------------------------------------
Sender Window : [Frame 4 - Frame 7]
Node A --> Sending Frame 5
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Received Frame 4, sending ACK 4
Node B --> Received Frame 5, sending ACK 5
Node B --> Received Frame 6, sending ACK 6
Node B --> Received Frame 7, sending ACK 7
Node A --> ACK 7 received. Sliding window forward.

-----------------------------------------
All 8 frames successfully transmitted from Node A to Node B.
```

frame and any subsequent out of order frames

ii) the sender times out and retransmits all frames starting from the lost frame.

d.) Else (no loss in this batch):

i) the receiver accepts each frame in order and sends a cumulative acknowledgement for it.

ii) On recieving the Ack for the last frame sent, the sender slides its window forward:

base = nextSeq.

5. Repeat step 4 until all N frames have been transmitted and acknowledged.

6. Stop the program.

PROGRAM:-

```
def simulate(total_frames, window_size, lost_frame):
    base=0
    next_seq=0.
    while base < total_frames:
        window_end = min(base + window_size - 1, total_frames
        print(f"Sender window: [Frame {base} - Frame {window
                                                    _end}]")

        while next_seq < base + window_size and next_seq
                                                < total_frames:
            print(f"Node A --> Sending Frame {next_seq}")
            if next_seq == lost_frame:
```

```
        print(f"  (Frame {next_seq} LOST in transit!)")
        next_seq += 1
    if lost_frame is not None and base <= lost_frame <
                                        next_seq:
        print(f"Node B -> Frame {lost_frame} missing. Discarding
                                out-of-order frames after it.")
        lost_frame = None
    for f in range(base, next_seq):
        print(f"  Node B -> Received Frame {f}, sending ACK {f}")
    print(f"Node A -> ACK {next_seq-1} received. Sliding window
                                forward.")
    base = next_seq.

def main():
    print("  === SLIDING WINDOW PROTOCOL - FLOW CONTROL
                    SIMULATION (GO-BACK-N) ===\n")
    total_frames = int(input("Enter total number of
                    frames to send  : "))
    window_size = int(input("Enter sender's window
                                size  : "))
    lost_frame = int(input(f"Enter frame number
    to simulate as LOST in (0 to {total_frames-1},
                    enter -1 for no loss) : "))
    if lost_frame == -1:
        lost_frame = None
```

Exno:8
NMAP to discover live Host using Nmap Scans (ARP, ICMP, TCP) on the Try Hackme Platform.

Aim: To study the working of Nmap tool and discover live hosts in a network using different host discovery techniques such as ARP scanning, ICMP, TCP and UDP scanning on TryHackme Platform

Steps:

1.) Open the TryHackMe platform & select the Nmap room.

2.) Study the network topology and identify the source and destination hosts.

3.) Use ARP requested to discover the MAC Address for local Network.

4.) Observe how ARP packets are broadcast with the network a identity the devices then receive the request.

5.) Use the ICUP protocol to send ping request.

6.) Analyse the packet flow and identify the exchange of ARP packets.

7.) Perform ARP host discovery using the Nmap command & identify all active hosts in the local network.

ARP:

sudo nmap -PR -Sn 10.10.210.0/24

Starting Nmap

Nmap scan report for 10.10.210.1

Host is up

Nmap done.

ICMP:

nmap -PE -Sn 10.10.210.0/24

starting Nmap

Nmap scan report for 10.10.210.1

Host is up

Nmap done

TCP:

nmap -PS 23 -Sn 10.10.210.0/24

Sudo nmap -PA 80 -Sn 10.10.210.0/24

8.) Perform ICMP echo scanning and observe the host that respond to the scan.

9.) Perform ICMP timestamp and analyse the response generated by target system.

10.) Perform ICMP Address mark scanning & result with the previous scans.

11.) perform TCP scanning to identify live hosts by analysis SYN and Ack packets.

12.) Perform TCP Ack scanning to verify the accessibility of the target hosts.

13.) Perform TCP Ack scanning to verify how Nmap identifies active hosts through UDP packets.

14. perform a Reverse DNS lookups to obtain or hostname of the discovered system.

Result:-

Thus live hosts in the network were successfully discovered using ARP, ICMP UDP scanning techniques with the Nmap tool.

Exno: 9 Implementation of subnetting in cisco packet TRACER simulator.

a.) Design multiple subnets with a suitable number of host.

AIM:-
To design multiple subnets with a suitable number of hosts using subnetting techniques in cisco Packet Tracer.

steps:-

1.) open cisco packet tracer and create a new project.

2.) Design the network topology by planning the required network device.

3.) Select a suitable network address for the experiment

4.) calculate the number of subnets required.

5.) Determine the no. of host needed in each subnet.

6.) calculate the subnet mask using subnetting techniques.

7.) Divide the network into multiple subnets.

8. Assign a separate network address to each subnet.

IP configuration.

Static.

IP Address 192.168.1.11

Subnet mask 255.255.255.0

Default Gateway 192.168.1.1

DNS Server.

IPV6 configuration.

Link local Address FE80::2D0:BAFF:FE9C
:D809.

9) verify the subnet calculations and host ranges

10) save the network topology.

Result:-

Thus, multiple subnet were successfully designed according to the required number of hosts.

b) Assign static IP addresses across all subnets and connect the subnet through a router.

Aim:- to assign static IP address to all devices in different subnet & establish communication between the subnet through a router.

Steps:-

1.) connect a router in the switches using appropriate cables.

2.) configure the router interface.

3.) Assign an IP address & subnet mask to each other router.

4. configure the switches.

5.) Assign static IP address to all computer.

6.) configure the subnet mask for each device

7.) Assign appropriate default gateway to each computer.

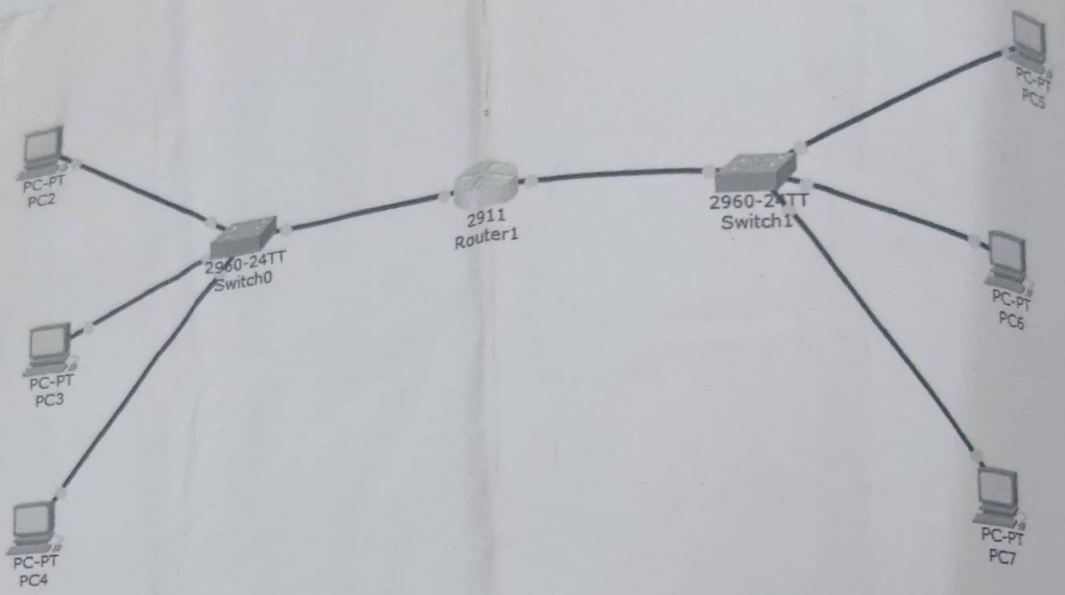
8.) verify the IP configuration.

9.) Save the router configuration

10.) verify the all devices are properly

Result:-

Thus, the static IP addresses were assi successfully and communicated between the subnets was established through router.



PC0

Physical Config Desktop Custom Interface

Command Prompt

```
Packet Tracer PC Command Line 1.0
PC>
Packet Tracer PC Command Line 1.0
PC>
Packet Tracer PC Command Line 1.0
PC>
Packet Tracer PC Command Line 1.0
PC>ping 192.168.0.1

Pinging 192.168.0.1 with 32 bytes of data:

Reply from 192.168.0.1: bytes=32 time=0ms TTL=255
Reply from 192.168.0.1: bytes=32 time=0ms TTL=255
Reply from 192.168.0.1: bytes=32 time=0ms TTL=255
Reply from 192.168.0.1: bytes=32 time=0ms TTL=255

Ping statistics for 192.168.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

PC>
```

c.) Stimulate packet transmission across the subnets and observe the result.

a.) When subnets are connected through a router.

AIM: To analyze packet transmission between different subnets when a router is used for communication.

Steps:-

1.) Open simulator mode in cisco packet tracer.

2.) Select a destination device from ~~another~~ subnet.

3.) Select a destination device from another subnet.

4.) Generate a packet using simple PDU tool.

5.) Observe movement of packet through the network.

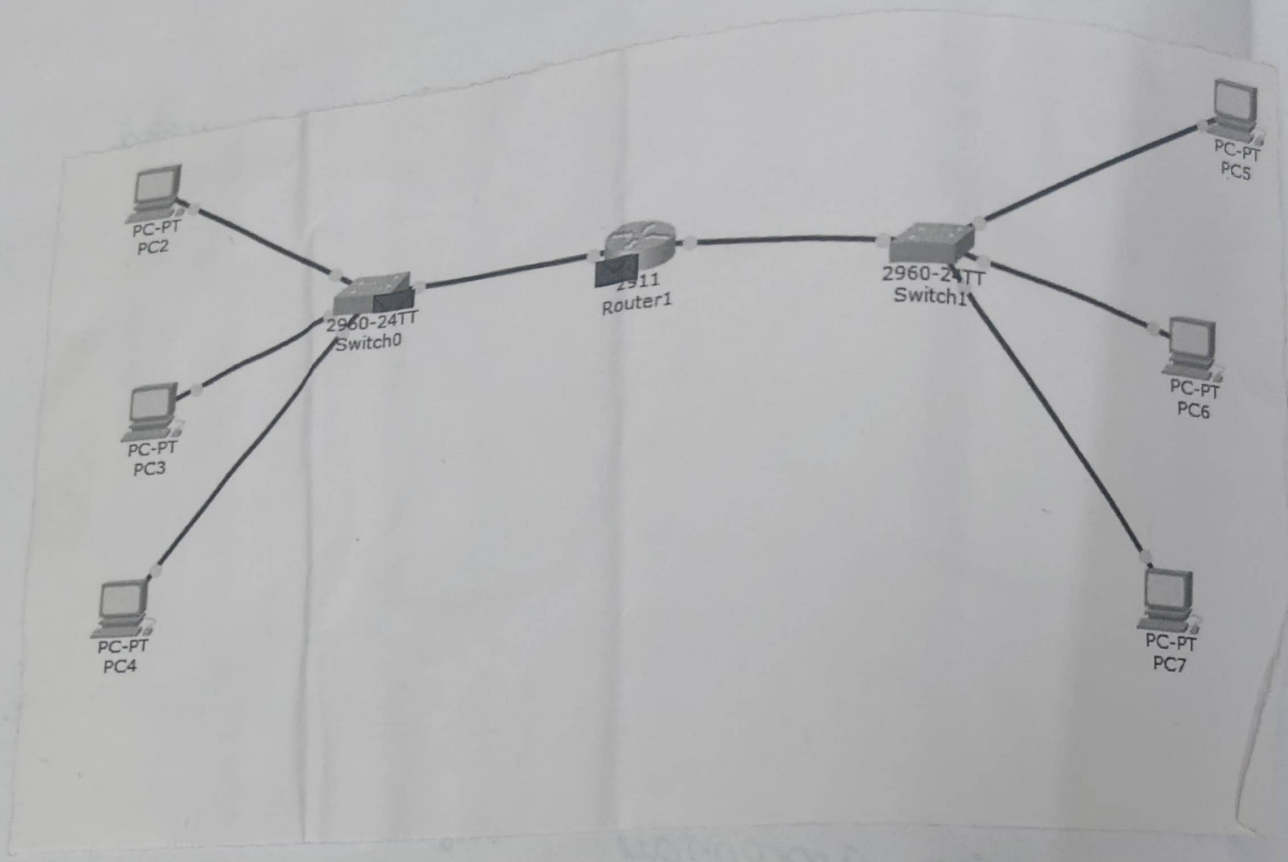
7.) Observer forwarding process performed by router.

8.) Verify the packet info between different subnets.

9.) Use the ping command to test connectivity

10.) Observe delivery of packets.

Result:- Thus packets were transmitted successfully between different subnets through the router.


PC-PT
PC2
PC-PT
PC3
PC-PT
PC4
2960-24TT
Switch0
2911
Router1
2960-24TT
Switch1
PC-PT
PC5
PC-PT
PC6
PC-PT
PC7

10/9/25

Exno:10.

Interaetworking with routers in cisco packet tracer simulator.

a.) Simple Internetwork using a router.

AIM:

To design and configure a simple internetwork using a router in cisco packet tracer and observe the routing of packets between different networks.

Algorithm:

1.) Open cisco Packet Tracer.

2. Place one cisco 2911 Router, two switches and 3-4 PCs for each network.

3.) Connect the PCs to the switches using copper straight.

4.) Connect the switches to the router

5.) Assign static IP addresses to all PCs and router interfaces.

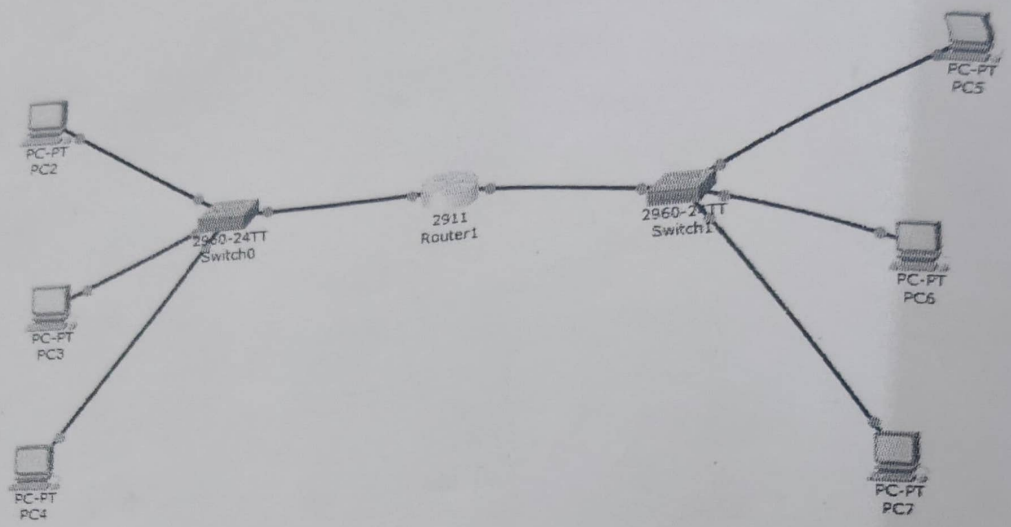
6.) Configure the router interfaces using the assigned IP addresses.

7.) Enable the router interfaces using

Show ip interface brief

9.) Use simple PDU or the ping command to send a packet from one network to another.

```
Router(config)#interface gigabitethernet 0/1
Router(config-if)#ip address 192.168.20.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state
to up
exit
```




```
Router#enable
Router#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.10.1    YES manual up                    up
GigabitEthernet0/1     192.168.20.1    YES manual up                    up
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
Router#
```

Copy Paste

10.) Select simulation mode and observe travelling through the switches and router

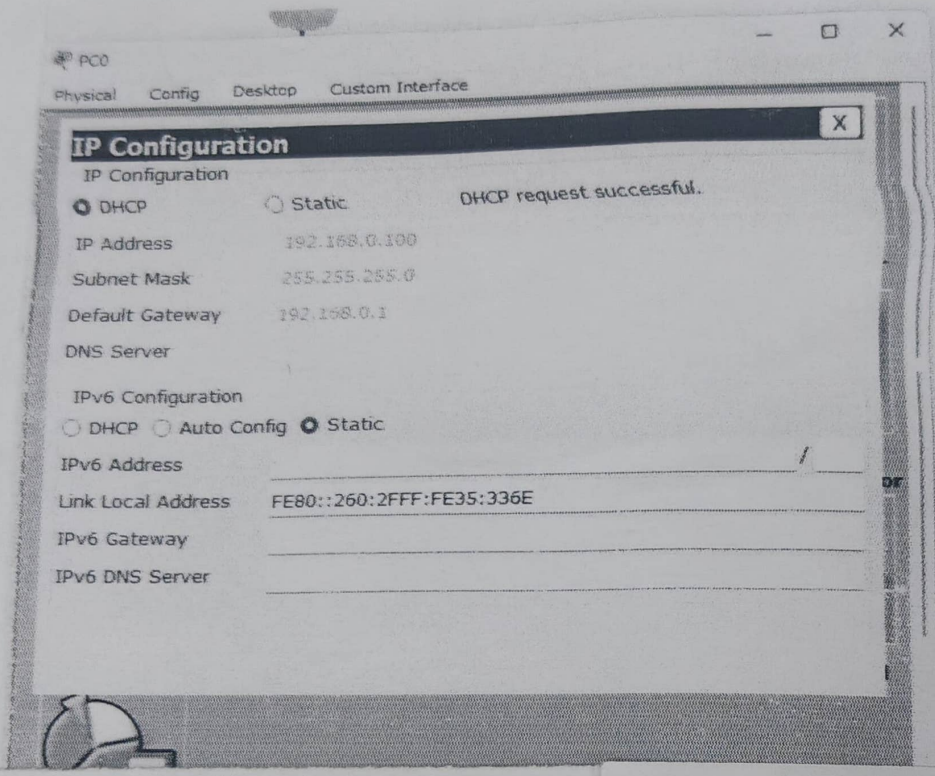
11.) Observe that the router forwards the packet through appropriate interface according to the destination network.

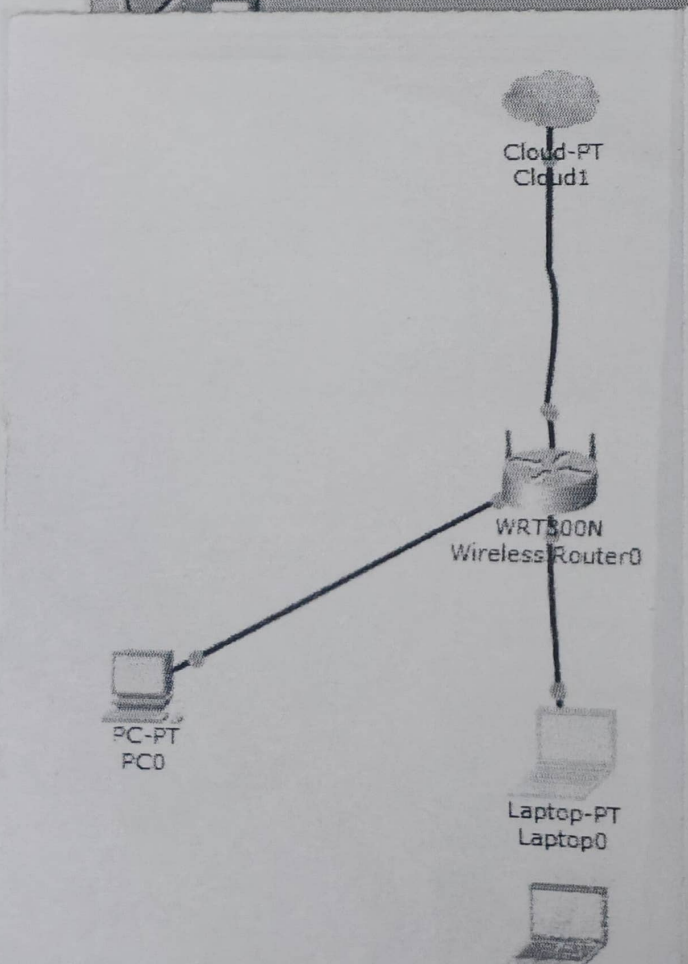
Result:

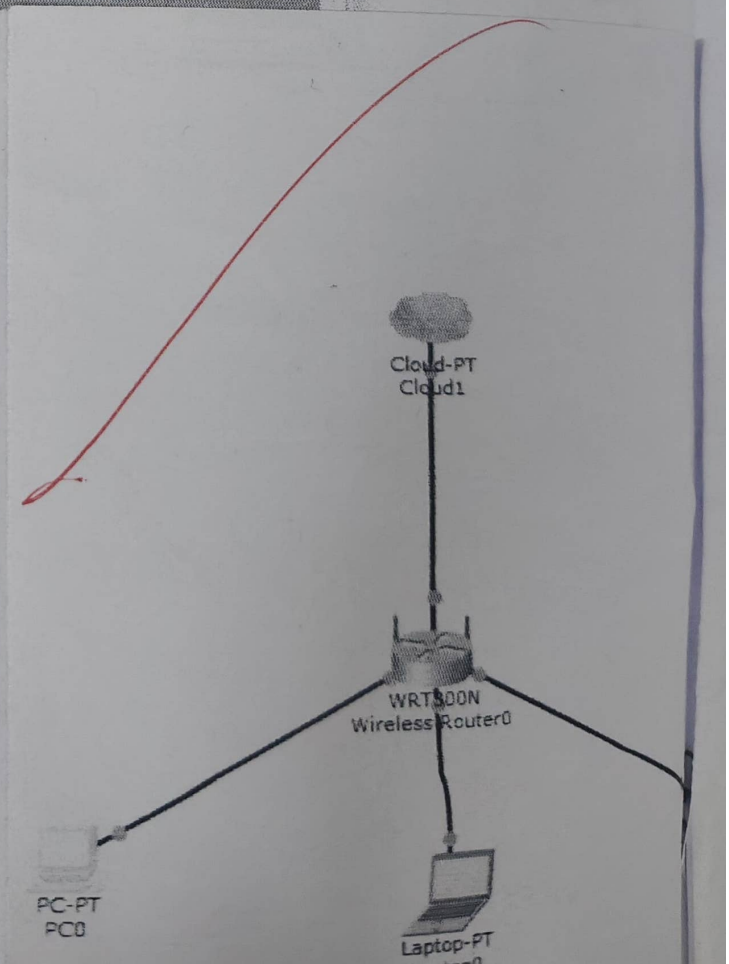
The simple internetwork was successfully designed and configured using a router in cisco packet tracer.

Packet transmission and routing between different networks were successfully observed.

2960-24TT
Switch0
Router1
2960-24TT
Switch1
PC-PT
PC3
PC-PT
PC6
PC-PT
PC4
PC-PT
PC7


PC0
Physical
Config
Desktop
Custom Interface
IP Configuration
X
IP Configuration
DHCP
Static
DHCP request successful.
IP Address
192.168.0.100
Subnet Mask
255.255.255.0
Default Gateway
192.168.0.1
DNS Server
IPv6 Configuration
DHCP
Auto Config
Static
IPv6 Address
Link Local Address
FE80::260:2FFF:FE35:336E
IPv6 Gateway
IPv6 DNS Server


Cloud-PT
Cloud1
WRT300N
Wireless Router0
PC-PT
PC0
Laptop-PT
Laptop0


Cloud-PT
Cloud1
WRT300N
Wireless Router0
PC-PT
PC0
Laptop-PT

b.) wireless router with DHCP server and internet cloud.

Aim: To design and configure an internetwork using a wireless router.

Algorithm:

1.) Open cisco packet tracer.

2.) place one cloud-PT, one WRT300N wireless router, 1PC, 2 laptops.

3.) connect the PC to a LAN port of WRT300N.

4. connect the cloud PT to internet/ WAN port of WRT300N.

5. configure the WRT 300N with LAN IP address and subnet mask.

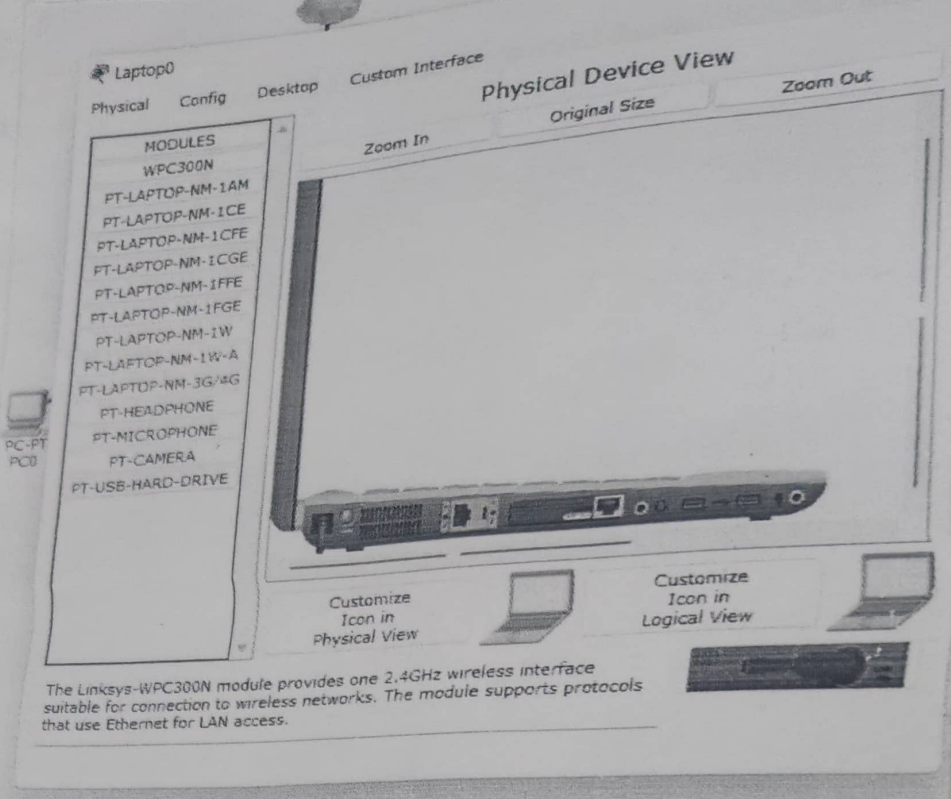
7. Enable the DHCP server in the wireless router.

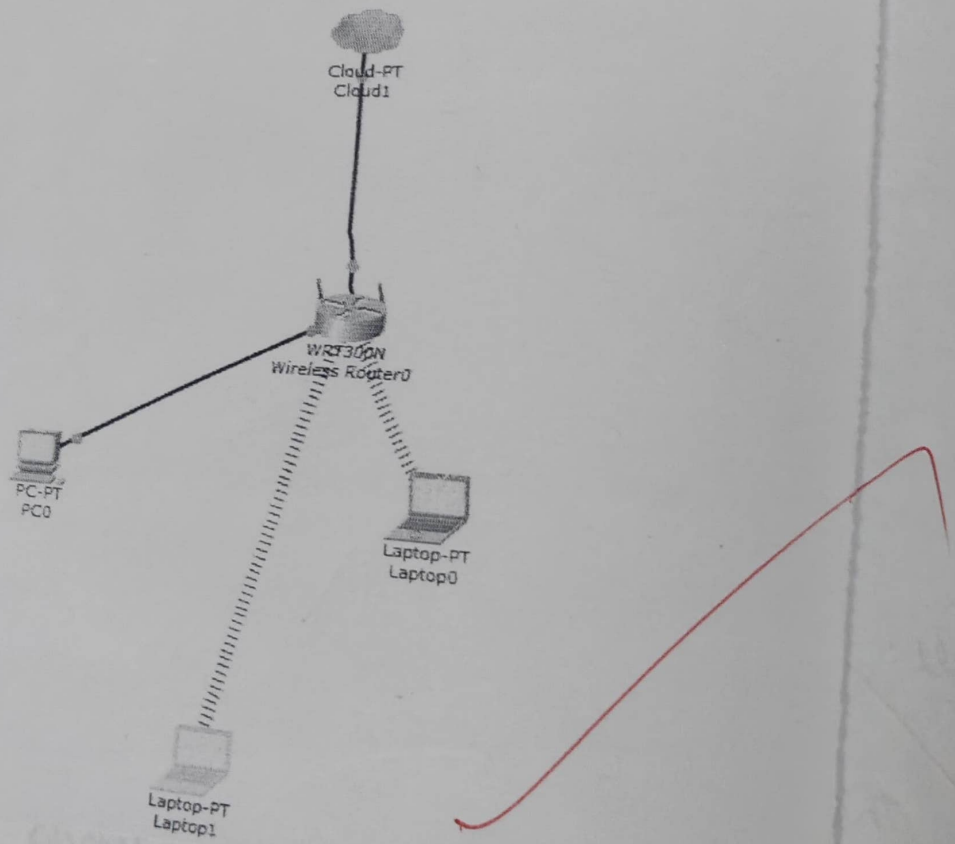
8. configure the PC/laptop to obtain their IP addresses.

9. connect the laptops to the wireless network of WRT 300N.

10. verify that the devices have received IP addresses

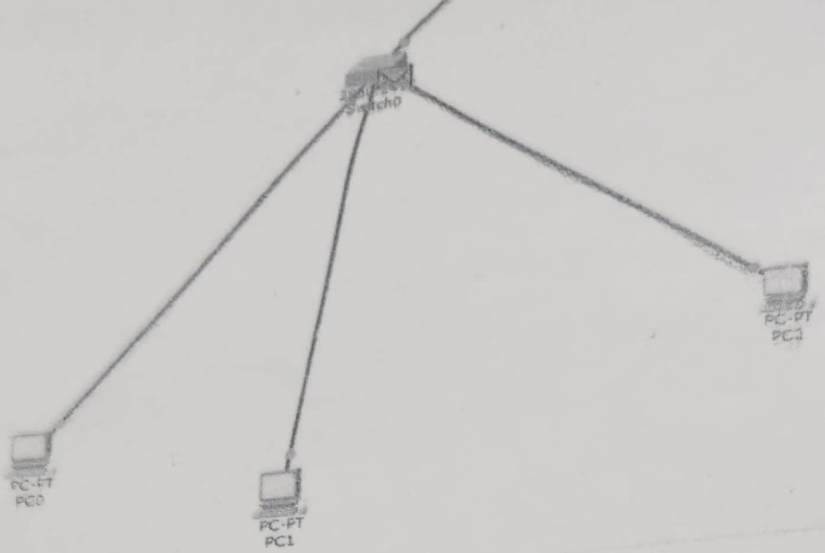
11. Use the ping command to test connectivity between the devices.


Laptop0
Physical
Config
Desktop
Custom Interface
Physical Device View
Zoom In
Original Size
Zoom Out
MODULES
WPC300N
PT-LAPTOP-NM-1AM
PT-LAPTOP-NM-1CE
PT-LAPTOP-NM-1CFE
PT-LAPTOP-NM-1CGE
PT-LAPTOP-NM-1FFE
PT-LAPTOP-NM-1FGE
PT-LAPTOP-NM-1W
PT-LAPTOP-NM-1W-A
PT-LAPTOP-NM-3G/4G
PT-HEADPHONE
PT-MICROPHONE
PT-CAMERA
PT-USB-HARD-DRIVE
PC-PT
PC0
Customize
Icon in
Physical View
Customize
Icon in
Logical View
The Linksys-WPC300N module provides one 2.4GHz wireless interface suitable for connection to wireless networks. The module supports protocols that use Ethernet for LAN access.

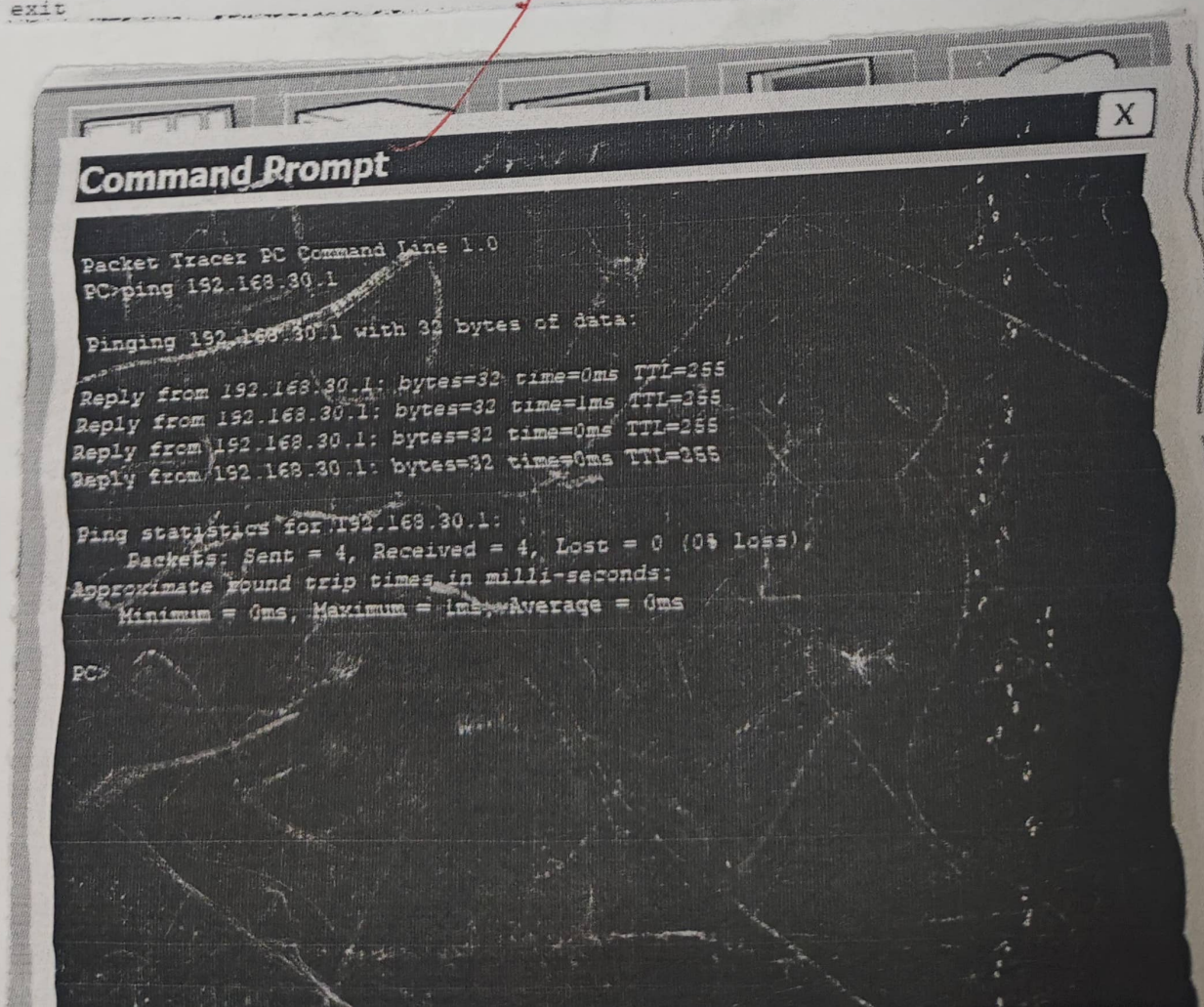

Cloud-PT
Cloud1
WRT300N
Wireless Router0
PC-PT
PC0
Laptop-PT
Laptop0
Laptop-PT
Laptop1

Results:

The internetwork was successfully designed and configurated. The devices obtained IP addresses automatically through DHCP and communication.




```
        --- System Configuration Dialog ---

Continue with configuration dialog? [yes/no]: no


Press RETURN to get started!


Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface gigabitethernet 0/0
Router(config-if)#ip address 192.168.10.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state
to up
exit
```



10/9/25 EXNO:11 Routing at Network layer.

Aim:
To stimulate and configure static routing protocol using cisco packet tracer

Procedure:

1. Open cisco packet tracer
2. Place three routers, two switches and four PCs
3. Connect the devices using appropriate ethernet cables.
4. Configure IP Addresses of the router interfaces and PCs.
5. Enable the router interfaces using the no shutdown command.
6. Configure static routes on the routers using the ip route command
7. Configure a primary & backup route.
8. Verify the routing table.
10. Stimulate failure of main route & verify that backup route is used.

Result:

Thus, static routing was successfully stimulated & configured using cisco packet tracer.

outputs:

Router # show ip route static

S 30.0.0.0/8 [10/10] via 20.0.0.2

S 30.0.0.100/32 [10/10] via 40.0.0.2

S 50.0.0.0/8 [10/10] via 40.0.0.2

Router 1

Router # show ip router static.

S 10.0.0.0/8 [10/10] via 20.0.0.1

S 40.0.0.0/8 [10/10] via 20.0.0.1

b.) RIP Routing:

Aim: To stimulate RIP using cisco packet tracer and observe dynamic routing between different networks.

Procedure:

1. Open cisco packet traur
2. plau 3 routers, 2 switches, UPCs.
3. connect devices using appropriate ethernet cables.
4. configure the IP addresses.
5. Enable the router interface.
6. configure static routes on the routers using the IP route command.
7. configure a primary route a backup rou
8. verify routing table.
9. Stimulate failure of main route

Verify that the backup route is used this is demonstrated by removing the main route and checking alternate path.

Result:

Thus RIP was successfully stimulated using cisco packet tracer and dynamic route selection and automatic routing were verified.

ping 10.0.0.2

pinging 10.0.0.2 with 32 bytes of data

Request timed out

Reply from 10.0.0.2 bytes = 32 time
= 3ms TTL = 126

Reply from 10.0.0.2 bytes = 32 time = 3ms
TTL = 126

Ping statistics for 10.0.0.2

packets sent = 4, received = 3, lost = 1

Approximate round trip time in milliseconds

min = 3ms, max = 3ms,
Avg = 3ms

7/9/24

Exno:12 Implementation of Client Server.

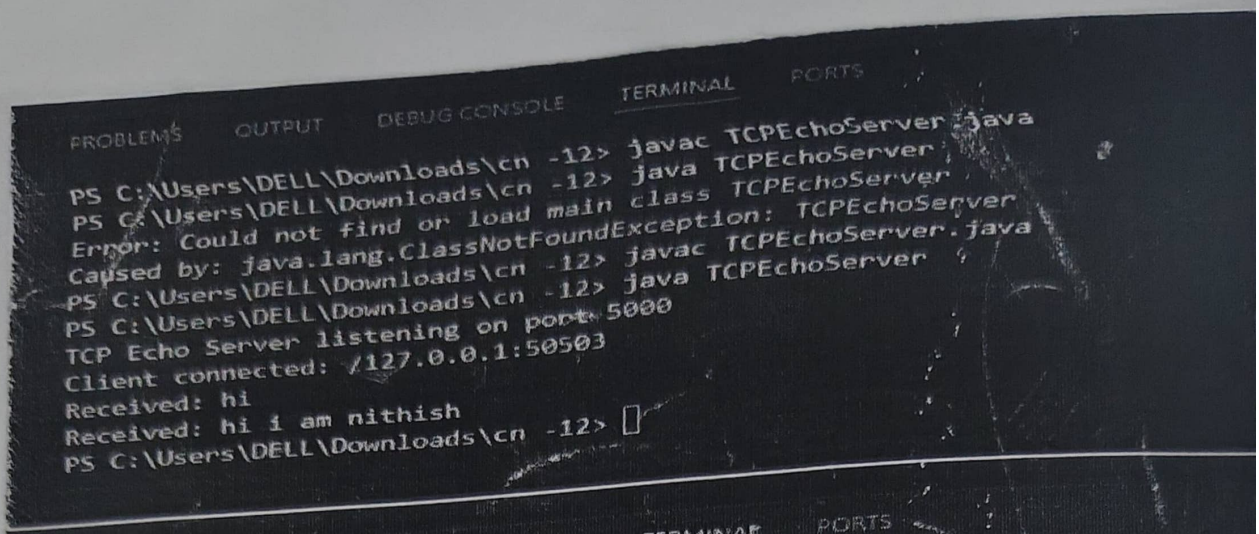
a.) Echo Client Server using TCP.

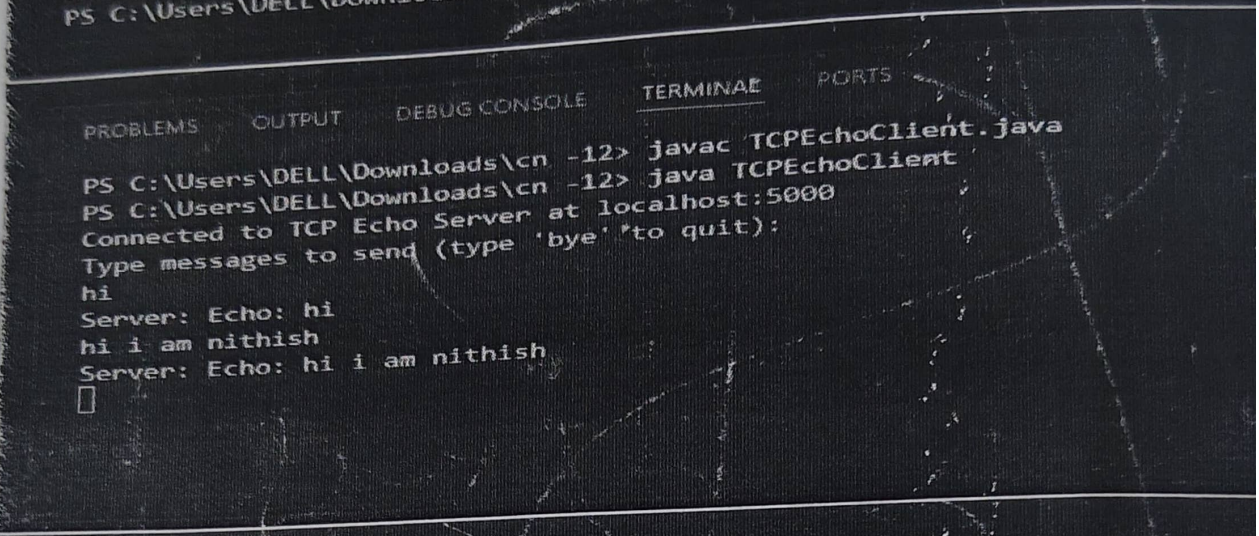
Aim:
To implement an Echo client server application using TCP sockets and verify communication between the client and server.

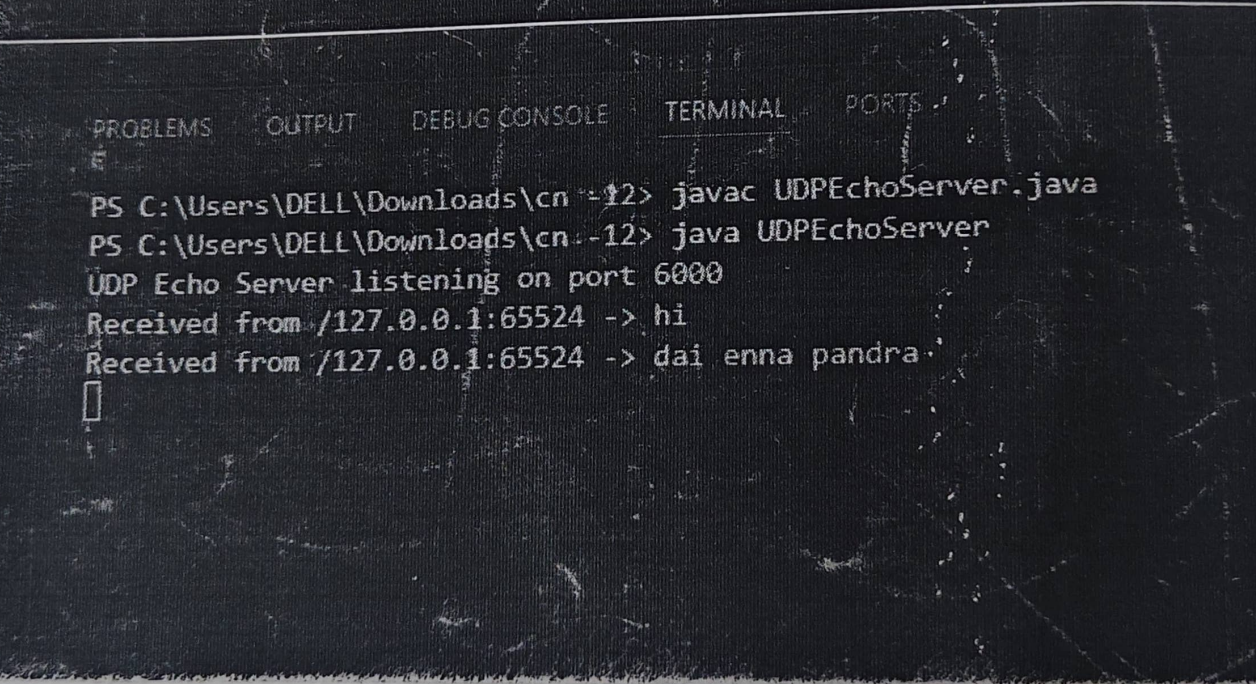
Procedure:
1. Create a TCP Echo server using Server Server Socket and socket.
2. Assign a port number to the server.
3. Compile and execute the TCP server program.
4. Create a TCP Echo client using socket.
5. Connect the client to the server using the server address and port number.
6. Enter a message through the client.
7. The server receives the message and sends it back to the client.
8. Display the echoed message at the client.
9. Enter bye to terminate the communication.

Code:. TCPEchoServer.java.

```
import java.io.*;
import java.net.*;
public class TCPEchoServer {
public static void main(String [] args)
throws Exception {
int port = args.length > 0 ? Integer.
parseInt (args[0]):
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\DELL\Downloads\cn -12> javac TCPEchoServer.java
PS C:\Users\DELL\Downloads\cn -12> java TCPEchoServer
Error: Could not find or load main class TCPEchoServer
Caused by: java.lang.ClassNotFoundException: TCPEchoServer
PS C:\Users\DELL\Downloads\cn -12> javac TCPEchoServer.java
PS C:\Users\DELL\Downloads\cn -12> java TCPEchoServer
TCP Echo Server listening on port 5000
Client connected: /127.0.0.1:50503
Received: hi
Received: hi i am nithish
PS C:\Users\DELL\Downloads\cn -12>


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\DELL\Downloads\cn -12> javac TCPEchoClient.java
PS C:\Users\DELL\Downloads\cn -12> java TCPEchoClient
Connected to TCP Echo Server at localhost:5000
Type messages to send (type 'bye' to quit):
hi
Server: Echo: hi
hi i am nithish
Server: Echo: hi i am nithish


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\DELL\Downloads\cn -12> javac UDPEchoServer.java
PS C:\Users\DELL\Downloads\cn -12> java UDPEchoServer
UDP Echo Server listening on port 6000
Received from /127.0.0.1:65524 -> hi
Received from /127.0.0.1:65524 -> dai enna pandra

```
try (ServerSocket serverSocket = new ServerSocket (port)) {
    System.out.println ("Client connected : " + clientSocket.
                        getRemoteSocketAddress());
    BufferedReader in = new BufferedReader (
                        new InputStreamReader (clientSocket.
                        getInputStream()));
    PrintWriter out = new PrintWriter (Client
    Socket.getOutputStream(), true);

    String line;
    while ((line = in.readLine()) != null) {
        System.out.println ("Received: " + line);
        out.println ("Echo: " + line);
        if (line.equalIgnoreCase ("bye")) {
            break;
        }
    }
    System.out.println ("Client disconnected: " +
        clientSocket.getRemoteSocketAddress());
} catch (IOException e) {
    System.out.println ("Connection error: " + e
                        .getMessage());
}
}
```

Result:

~~Thus, the TCP Echo client server application~~ ~~successfully implemented, and the client~~

TCPEchoClient.java

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\Downloads\cn -12> javac UDPEchoClient.java
PS C:\Users\DELL\Downloads\cn -12> java UDPEchoClient
UDP Echo Client ready to send to localhost:6000
Type messages to send (type 'bye' to quit):
hi
Server: Echo: hi
dai enna pandra
Server: Echo: dai enna pandra
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
        at java.base/java.net.ServerSocket.<init>(ServerSocket.java:138)
        at ChatServer.main(ChatServer.java:12)
PS C:\Users\DELL\Downloads\cn -12> netstat -ano | findstr :5000
  TCP    0.0.0.0:5000           0.0.0.0:0              LISTENING       10340
  TCP    127.0.0.1:5000         127.0.0.1:50503        ESTABLISHED     10340
  TCP    127.0.0.1:50503        127.0.0.1:5000         ESTABLISHED     2852
  TCP    [::]:5000              [::]:0                 LISTENING       10340
PS C:\Users\DELL\Downloads\cn -12> taskkill /PID 10340 /F
SUCCESS: The process with PID 10340 has been terminated.
PS C:\Users\DELL\Downloads\cn -12> java ChatServer
Chat server started on port 5000
nithish connected from /127.0.0.1
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\Downloads\cn -12> javac ChatClient.java
PS C:\Users\DELL\Downloads\cn -12> java ChatClient
Enter your name:
nithish
Welcome, nithish! Type /quit to exit.
```

```
import java.io.*;
import java.net.*;

public class TCP Echo client {
    public static void main (String [] args) throws
    IOException {
        String host = args.length > 0 ? args
                                [0] : "local host";
    int port = args.length > 1 ? Integer.parseInt(
                                args[1]) : 5000;

    try (Socket socket = new Socket (host, port);
    PrintWriter out = new PrintWriter (socket.
                    getOutputStream (), true);
    BufferedReader in = new BufferedReader (
                    new InputStreamReader (socket.
                    getInputStream ()));
    BufferedReader stdIn = new BufferedReader
    (new InputStreamReader (System.in)))}

    System.out.println ("connected to TCP Echo server
                    at" + host + ":" + port);

    System.out.println ("type messages to
                    send (type 'bye' to quit): ");

    String user input:
```

```
while ((userInput = stdIn.readLine()) !=
                                    null)}

    out.println(userInput);
    string response = in.readline();
    system.out.println("server:" + response);
if (user.equal Ignore case ("bye".)) {
        break;
    }
  }
}
  System.out.println("connection closed.");
  }
}
```

Result:

Thus the TCP Echo client server Application was successfully implemented and client successfully received the echoed msg from the server.

Exno:13 Implement your own ping program.

DATE: 24.9.26

Aim:

To implement a simple ping program in computer networks to check whether a destination host is reachable and to measure the round-trip time of packets.

Algorithm:

Start the program.
Read the destination host name or IP address from the user
Create a raw ICMP socket
Construct an ICMP Echo request packet containing: ICMP type
code
checksum
Identifier
sequence number.
Calculate checksum of the packet.
Wait for an ICMP Echo reply.
Display the destination, packet size, sequence number and RTT
Display the final result and terminate.

program:

```
import socket
import struct
import time
import os

def checksum(data):
    total = 0
    length = len(data)
```

```
for i in range(0, length-1, 2):
        total += (data[i]<<8)+data[i+1]

total = (total >>16) + (total & 0xffff)
total += total >>16
return (~total) & 0xffff

def ping(host):
try:
    dest_addr = socket.gethostbyname(host)
except socket.gaierror
    print("Unable to resolve host")
    return.

try:
    sock = socket.socket(socket.AF_INET, socket.SOCK_RAW
                         icmp)
except permission Error:
    print("permission denied. Run the program
    with administrator.").
    return.

header = struct.pack("!BBHHH", 8, 0, 0, packet_id, sequence)
data = b" computer networks ping"
packet = header + data
Chksum = checksum(packet)

host = input("Enter hostname or IP address: ")
for i in range(4):
    ping(host)
    time.sleep(1).
```

~~Sam~~ Output:

Enter host name or IP address : 8.8.8.8

Reply from 8.8.8.8 bytes = 43 time = 24.51 ms

Reply from 8.8.8.8 bytes = 43 time = 23.87 ms

Reply from 8.8.8.8 bytes = 43 time = 24.12 ms

Reply from 8.8.8.8 bytes = 43 time = 24.36 ms.

Result:-

Thus, a ping program was successfully implemented using python. The program sends ICMP Echo Request packets to the specified host, receives echo reply packets, and calculates the RTT.

Exno:14. Write a code using RAW sockets
Date: 2/10/26 to implement packet sniffing.

Aim:- To write a program using RAW sockets to capture and display network packets passing the network interface.

Algorithm:-

Start the program.

Create a raw socket.

Bind the socket to the required network interface.

Continuously receive packets from the network.

Extract the Ethernet header from each packet

For IPv4 packets, extract the source.

Display packet information.

Repeat until user terminates

Close the socket.

Program:-

```
import socket
import struct

raw_socket = socket.socket(
        socket.AF_PACKET,
        socket.SOCK_RAW,
        socket.ntohs(3)
)
```

```
try:
while True:
    packet, address = raw_socket.recvfrom(65535)
    ethernet_header = packet[:14]
    dest_mac, src_mac, protocol = struct.unpack(
        "!6s6sH", ethernet_header
    )
    print("Source MAC :", src_mac)
    print("Destination MAC :", dest_mac)
    print("Protocol :", hex(protocol))

    version_ihl, tos, total_length, identification,
        flags_fragment, ttl, proto, checksum,
    src_ip, dest_ip = struct.unpack(
        "!BBHHHBBH4s4s",
        ip_header
    )
    print("Source IP :", src_ip)
    print("Destination IP :", dest_ip)

    print("TTL :", ttl)
    print("IP protocol :", proto)

elif protocol == 0x0806:
        print("Packet Type : ARP")
elif protocol == 0x86DD:
        print("Packet Type : IPv6")

else:
    print("Packet Type : Other")

finally:
    raw_socket.close().
```

Output:

Packet sniffer started...
waiting for packets...

source MAC: 3C:52:82:11:22:33
Destination MAC: ff:ff:ff:ff:ff:ff
protocol: 0x806
packet type: ARP.

source MAC: 3C:52:82:11:22:33.
Destination MAC: 84:16:f9:44:55:66.
protocol: 0x800
Source IP: 192.168.1.10
Destination IP: 192.168.1.1
TTL: 64
IP protocol: 6.

Source MAC: 84:16:f9:44:55:66
Destination MAC: 3C:52:82:11:22:33
protocol: 0x800
Source IP: 192.168.1.1
Destination IP: 192.168.1.10
TTL: 64

Result:-

Thus, a packet sniffing program using RAW sockets was successfully implemented. The program captures network packets such as MAC address, destination MAC address, protocol, source IP address.

Ex:15 Analyse various types of servers using webalizer tool.

Date: 8/10/24

Aim:

To analyze the performance and usage statistics of a web server using the webalizer log analysis tool.

Algorithm:

Start the system and install the webalizer tool.

Obtain the web server access log file, such as Apache's access.log.

Run webalizer to process the web server log.

Webalizer reads and analyzes the HTTP request records.

Generated the HTML report

Open the generated report in a web browser

Analyze the statistics and determine the server usage and performance.

Stop the program.

Procedure:

Webalizer is a log-analysis tool, so the practical implementation consists of configuring and executing webalizer.

Ex:15 Analyse various types of servers using webalizer tool.

Date: 8/10/24

Aim:

To analyze the performance and usage statistics of a web server using the webalizer log analysis tool.

Algorithm:

Start the system and install the webalizer tool.

Obtain the web server access log file such as Apache's access.log.

Run webalizer to process the web server log.

Webalizer reads and analyzes the HTTP request records.

Generated the HTML report.

Open the generated report in a web browser

Analyze the statistics and determine the server usage and performance.

Stop the program.

Procedure:

Webalizer is a log-analysis tool, so the practical implementation consists of configuring and executing webalizer.

Result:-

Thus, the webalizer tool was successfully used to analyze web server log files.

The generated report gives useful information which helps in evaluating web server usage.